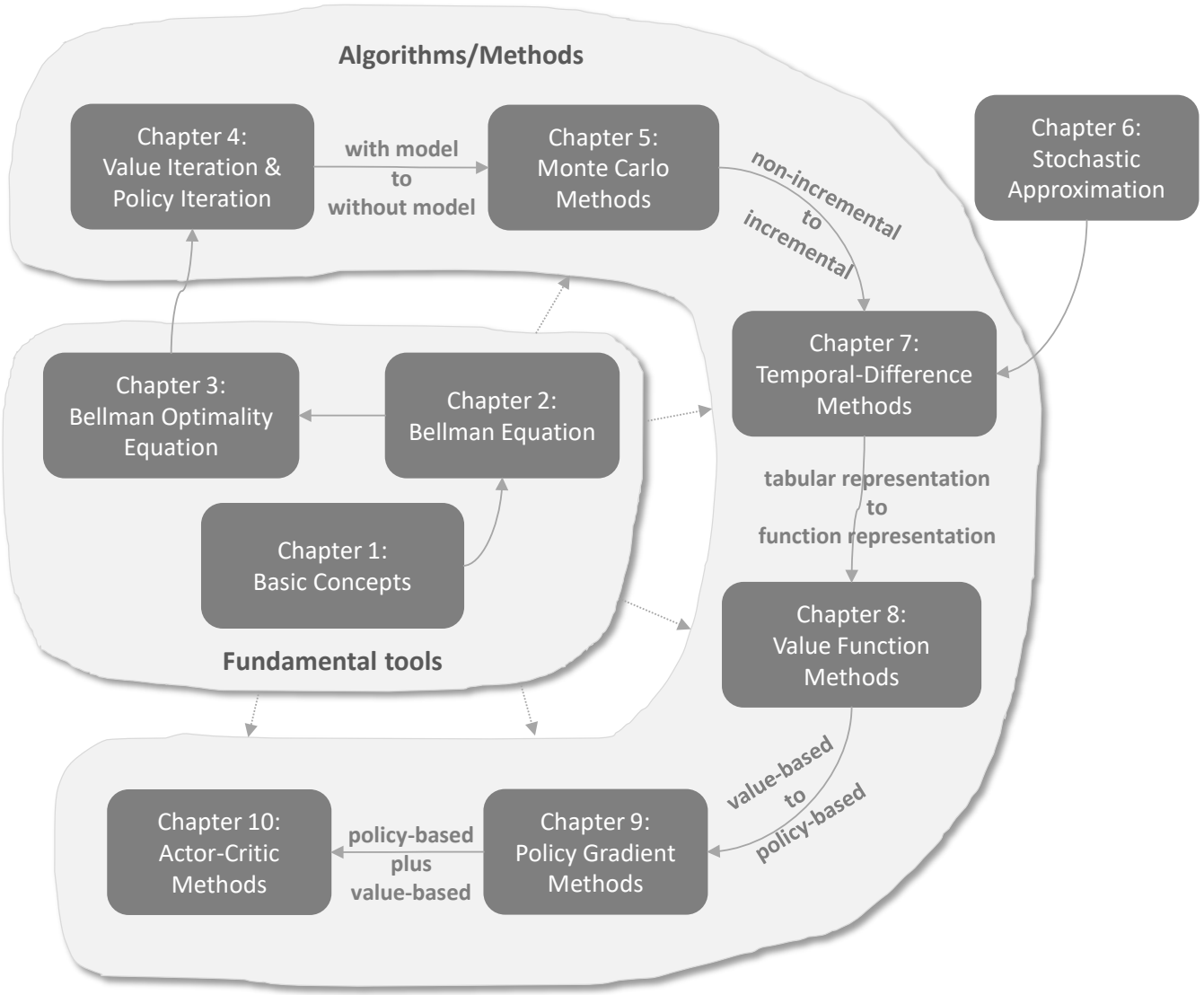
第七讲：时序差分方法

April 11, 2026

大纲


Algorithms/Methods
Chapter 4: Value Iteration & Policy Iteration
with model to without model
Chapter 5: Monte Carlo Methods
non-incremental to incremental
Chapter 6: Stochastic Approximation
Chapter 7: Temporal-Difference Methods
tabular representation to function representation
Chapter 8: Value Function Methods
value-based to policy-based
Chapter 9: Policy Gradient Methods
policy-based plus value-based
Chapter 10: Actor-Critic Methods
Chapter 3: Bellman Optimality Equation
Chapter 2: Bellman Equation
Chapter 1: Basic Concepts
Fundamental tools

介绍

- 本讲介绍**时序差分（TD）学习**，这是强化学习（RL）中最著名的方法之一。
- 蒙特卡洛（MC）学习是第一种无模型方法，TD 学习是第二种无模型方法，且TD 相比MC 具有一些优势。
- 我们将看到上一讲中学习的随机近似方法在这里是如何发挥作用的。

大纲

1 动机示例

2 状态值的TD 学习

3 动作值的TD 学习：Sarsa

4 动作值的TD 学习：n 步Sarsa

5 最优动作值的TD 学习：Q 学习

6 统一视角

7 总结

大纲

1 动机示例

2 状态值的TD 学习

3 动作值的TD 学习：Sarsa

4 动作值的TD 学习：n 步Sarsa

5 最优动作值的TD 学习：Q 学习

6 统一视角

7 总结

动机示例：随机算法

我们考虑一些随机问题，并展示如何使用RM 算法来求解它们。

第一，回顾均值估计问题： 计算

$$w = \mathbb{E}[X]$$

基于X 的独立同分布样本$\{x\}$。我们在上一讲中已经研究过该问题。

- 令$g(w) = w - \mathbb{E}[X]$，将问题转化为求根问题：

$$g(w) = 0$$

- 由于我们只能获得X 的样本$\{x\}$，带噪声的观测为：

$$\tilde{g}(w, \eta) = w - x = (w - \mathbb{E}[X]) + (\mathbb{E}[X] - x) \doteq g(w) + \eta$$

- 根据上一讲，我们知道求解$g(w) = 0$ 的RM 算法为：

$$w_{k+1} = w_k - \alpha_k \tilde{g}(w_k, \eta_k) = w_k - \alpha_k (w_k - x_k)$$

动机示例：随机算法

我们考虑一些随机问题，并展示如何使用RM 算法来求解它们。

第一，回顾均值估计问题： 计算

$$w = \mathbb{E}[X]$$

基于X 的独立同分布样本$\{x\}$。我们在上一讲中已经研究过该问题。

- 令$g(w) = w - \mathbb{E}[X]$，将问题转化为求根问题：

$$g(w) = 0$$

- 由于我们只能获得X 的样本$\{x\}$，带噪声的观测为：

$$\tilde{g}(w, \eta) = w - x = (w - \mathbb{E}[X]) + (\mathbb{E}[X] - x) \doteq g(w) + \eta$$

- 根据上一讲，我们知道求解$g(w) = 0$ 的RM 算法为：

$$w_{k+1} = w_k - \alpha_k \tilde{g}(w_k, \eta_k) = w_k - \alpha_k (w_k - x_k)$$

动机示例：随机算法

我们考虑一些随机问题，并展示如何使用RM 算法来求解它们。

第一，回顾均值估计问题： 计算

$$w = \mathbb{E}[X]$$

基于X 的独立同分布样本$\{x\}$。我们在上一讲中已经研究过该问题。

- 令$g(w) = w - \mathbb{E}[X]$，将问题转化为求根问题：

$$g(w) = 0$$

- 由于我们只能获得X 的样本$\{x\}$，带噪声的观测为：

$$\tilde{g}(w, \eta) = w - x = (w - \mathbb{E}[X]) + (\mathbb{E}[X] - x) \doteq g(w) + \eta$$

- 根据上一讲，我们知道求解$g(w) = 0$ 的RM 算法为：

$$w_{k+1} = w_k - \alpha_k \tilde{g}(w_k, \eta_k) = w_k - \alpha_k (w_k - x_k)$$

动机示例：随机算法

我们考虑一些随机问题，并展示如何使用RM 算法来求解它们。

第一，回顾均值估计问题： 计算

$$w = \mathbb{E}[X]$$

基于X 的独立同分布样本$\{x\}$。我们在上一讲中已经研究过该问题。

- 令$g(w) = w - \mathbb{E}[X]$，将问题转化为求根问题：

$$g(w) = 0$$

- 由于我们只能获得X 的样本$\{x\}$，带噪声的观测为：

$$\tilde{g}(w, \eta) = w - x = (w - \mathbb{E}[X]) + (\mathbb{E}[X] - x) \doteq g(w) + \eta$$

- 根据上一讲，我们知道求解$g(w) = 0$ 的RM 算法为：

$$w_{k+1} = w_k - \alpha_k \tilde{g}(w_k, \eta_k) = w_k - \alpha_k (w_k - x_k)$$

动机示例：随机算法

我们考虑一些随机问题，并展示如何使用RM 算法来求解它们。

第一，回顾均值估计问题： 计算

$$w = \mathbb{E}[X]$$

基于X 的独立同分布样本$\{x\}$。我们在上一讲中已经研究过该问题。

- 令$g(w) = w - \mathbb{E}[X]$，将问题转化为求根问题：

$$g(w) = 0$$

- 由于我们只能获得X 的样本$\{x\}$，带噪声的观测为：

$$\tilde{g}(w, \eta) = w - x = (w - \mathbb{E}[X]) + (\mathbb{E}[X] - x) \doteq g(w) + \eta$$

- 根据上一讲，我们知道求解$g(w) = 0$ 的RM 算法为：

$$w_{k+1} = w_k - \alpha_k \tilde{g}(w_k, \eta_k) = w_k - \alpha_k (w_k - x_k)$$

动机示例：随机算法

第二，考虑一个稍复杂的问题。 即估计函数$v(X)$ 的均值：

$$w = \mathbb{E}[v(X)],$$

基于X 的独立同分布随机样本$\{x\}$。

- 为求解该问题，定义：

$$g(w) = w - \mathbb{E}[v(X)]$$
$$\tilde{g}(w,\eta) = w - v(x) = (w - \mathbb{E}[v(X)]) + (\mathbb{E}[v(X)] - v(x)) \doteq g(w) + \eta.$$

- 该问题转化为求根问题：$g(w) = 0$。对应的RM 算法为：

$$w_{k+1} = w_k - \alpha_k \tilde{g}(w_k, \eta_k) = w_k - \alpha_k [w_k - v(x_k)]$$

动机示例：随机算法

第二，考虑一个稍复杂的问题。 即估计函数$v(X)$ 的均值：

$$w = \mathbb{E}[v(X)],$$

基于X 的独立同分布随机样本$\{x\}$。

- 为求解该问题，定义：

$$\begin{aligned} g(w) &= w - \mathbb{E}[v(X)] \\ \tilde{g}(w,\eta) &= w - v(x) = (w - \mathbb{E}[v(X)]) + (\mathbb{E}[v(X)] - v(x)) \doteq g(w) + \eta. \end{aligned}$$

- 该问题转化为求根问题：$g(w) = 0$。对应的RM 算法为：

$$w_{k+1} = w_k - \alpha_k \tilde{g}(w_k, \eta_k) = w_k - \alpha_k [w_k - v(x_k)]$$

动机示例：随机算法

第二，考虑一个稍复杂的问题。 即估计函数$v(X)$ 的均值：

$$w = \mathbb{E}[v(X)],$$

基于X 的独立同分布随机样本$\{x\}$。

- 为求解该问题，定义：

$$\begin{aligned} g(w) &= w - \mathbb{E}[v(X)] \\ \tilde{g}(w,\eta) &= w - v(x) = (w - \mathbb{E}[v(X)]) + (\mathbb{E}[v(X)] - v(x)) \doteq g(w) + \eta. \end{aligned}$$

- 该问题转化为求根问题：$g(w) = 0$。对应的RM 算法为：

$$w_{k+1} = w_k - \alpha_k \tilde{g}(w_k, \eta_k) = w_k - \alpha_k [w_k - v(x_k)]$$

动机示例：随机算法

第三，考虑一个更复杂的问题： 计算

$$w = \mathbb{E}[R + \gamma v(X)],$$

其中R, X 为随机变量，γ 为常数，$v(\cdot)$ 为某函数。

- 假设我们可以获得X 和R 的样本$\{x\}$ 和$\{r\}$。定义：

$$\begin{aligned} g(w) &= w - \mathbb{E}[R + \gamma v(X)], \\ \tilde{g}(w, \eta) &= w - [r + \gamma v(x)] \\ &= (w - \mathbb{E}[R + \gamma v(X)]) + (\mathbb{E}[R + \gamma v(X)] - [r + \gamma v(x)]) \\ &\doteq g(w) + \eta. \end{aligned}$$

- 该问题转化为求根问题：$g(w) = 0$。对应的RM 算法为：

$$w_{k+1} = w_k - \alpha_k \tilde{g}(w_k, \eta_k) = w_k - \alpha_k [w_k - (r_k + \gamma v(x_k))]$$

该算法与后面介绍的TD 算法形式非常相似。

动机示例：随机算法

第三，考虑一个更复杂的问题： 计算

$$w = \mathbb{E}[R + \gamma v(X)],$$

其中R, X 为随机变量，γ 为常数，$v(\cdot)$ 为某函数。

- 假设我们可以获得X 和R 的样本$\{x\}$ 和$\{r\}$。定义：

$$\begin{aligned} g(w) &= w - \mathbb{E}[R + \gamma v(X)], \\ \tilde{g}(w, \eta) &= w - [r + \gamma v(x)] \\ &= (w - \mathbb{E}[R + \gamma v(X)]) + (\mathbb{E}[R + \gamma v(X)] - [r + \gamma v(x)]) \\ &\doteq g(w) + \eta. \end{aligned}$$

- 该问题转化为求根问题：$g(w) = 0$。对应的RM 算法为：

$$w_{k+1} = w_k - \alpha_k \tilde{g}(w_k, \eta_k) = w_k - \alpha_k [w_k - (r_k + \gamma v(x_k))]$$

该算法与后面介绍的TD 算法形式非常相似。

动机示例：随机算法

第三，考虑一个更复杂的问题： 计算

$$w = \mathbb{E}[R + \gamma v(X)],$$

其中R, X 为随机变量，γ 为常数，$v(\cdot)$ 为某函数。

- 假设我们可以获得X 和R 的样本$\{x\}$ 和$\{r\}$。定义：

$$\begin{aligned} g(w) &= w - \mathbb{E}[R + \gamma v(X)], \\ \tilde{g}(w, \eta) &= w - [r + \gamma v(x)] \\ &= (w - \mathbb{E}[R + \gamma v(X)]) + (\mathbb{E}[R + \gamma v(X)] - [r + \gamma v(x)]) \\ &\doteq g(w) + \eta. \end{aligned}$$

- 该问题转化为求根问题：$g(w) = 0$。对应的RM 算法为：

$$w_{k+1} = w_k - \alpha_k \tilde{g}(w_k, \eta_k) = w_k - \alpha_k[w_k - (r_k + \gamma v(x_k))]$$

该算法与后面介绍的TD 算法形式非常相似。

动机示例：随机算法

简要总结:

- 以上三个例子复杂度依次增加。
- 它们均可以用RM 算法求解。
- 我们将看到TD 算法具有类似的表达式。

大纲

1 动机示例

2 状态值的TD 学习

3 动作值的TD 学习：Sarsa

4 动作值的TD 学习：n 步Sarsa

5 最优动作值的TD 学习：Q 学习

6 统一视角

7 总结

状态值的TD 学习——算法描述

问题设定:

- 给定策略π，目标是估计π 下的状态值$\{v_\pi(s)\}_{s\in\mathcal{S}}$。
- 经验样本：$(s_0, r_1, s_1, \ldots, s_t, r_{t+1}, s_{t+1}, \ldots)$ 或$\{(s_t, r_{t+1}, s_{t+1})\}_t$，由策略$\pi$生成。

重要符号说明:

$$v(s) \longrightarrow v_\pi(s)$$

状态值的TD 学习——算法描述

问题设定:

- 给定策略π，目标是估计π 下的状态值$\{v_\pi(s)\}_{s\in\mathcal{S}}$。
- 经验样本：$(s_0, r_1, s_1, \ldots, s_t, r_{t+1}, s_{t+1}, \ldots)$ 或$\{(s_t, r_{t+1}, s_{t+1})\}_t$，由策略$\pi$生成。

重要符号说明:

$$v(s) \longrightarrow v_\pi(s)$$

状态值的TD 学习——算法描述

问题设定:

- 给定策略π，目标是估计π 下的状态值$\{v_\pi(s)\}_{s\in\mathcal{S}}$。
- 经验样本：$(s_0, r_1, s_1, \ldots, s_t, r_{t+1}, s_{t+1}, \ldots)$ 或$\{(s_t, r_{t+1}, s_{t+1})\}_t$，由策略$\pi$生成。

重要符号说明:

$$
\begin{array}{c}
v(s) \longrightarrow v_\pi(s) \\
\Downarrow \\
v(s_t) \longrightarrow v_\pi(s_t)
\end{array}
$$

状态值的TD 学习——算法描述

问题设定:

- 给定策略π，目标是估计π 下的状态值$\{v_\pi(s)\}_{s\in\mathcal{S}}$。
- 经验样本：$(s_0, r_1, s_1, \ldots, s_t, r_{t+1}, s_{t+1}, \ldots)$ 或$\{(s_t, r_{t+1}, s_{t+1})\}_t$，由策略$\pi$生成。

重要符号说明:

$$
\begin{array}{c}
v(s) \longrightarrow v_\pi(s) \\
\Downarrow \\
v(s_t) \longrightarrow v_\pi(s_t) \\
\Downarrow \\
v_t(s_t) \longrightarrow v_\pi(s_t)
\end{array}
$$

状态值的TD 学习——算法描述

TD 学习算法为：

$$v_{t+1}(s_t) = v_t(s_t) - \alpha_t(s_t)\Big[v_t(s_t) - [r_{t+1} + \gamma v_t(s_{t+1})]\Big], \tag{1}$$

$$v_{t+1}(s) = v_t(s), \quad \forall s \neq s_t, \tag{2}$$

其中$t = 0, 1, 2, \ldots$。

其中，$v_t(s_t)$ 是对$v_\pi(s_t)$ 的估计值；$\alpha_t(s_t)$ 是s_t 在时刻t 的学习率。

- 在时刻t，只有被访问状态s_t 的值会被更新，未被访问的状态$s \neq s_t$ 的值保持不变。
- 当上下文明确时，式(2) 将被省略。

状态值的TD 学习——算法性质

TD 算法可标注为：

$$\underbrace{v_{t+1}(s_t)}_{\text{新估计}} = \underbrace{v_t(s_t)}_{\text{当前估计}} - \alpha_t(s_t)\big[\overbrace{v_t(s_t) - \underbrace{[r_{t+1} + \gamma v_t(s_{t+1})]}_{\text{TD 目标}\bar{v}_t}}^{\text{TD 误差}\delta_t}\big], \tag{3}$$

其中，

$$\bar{v}_t \doteq r_{t+1} + \gamma v_t(s_{t+1})$$

称为TD 目标。

$$\delta_t \doteq v_t(s_t) - [r_{t+1} + \gamma v_t(s_{t+1})] = v_t(s_t) - \bar{v}_t$$

称为TD 误差。

观察：新估计$v_{t+1}(s_t)$ 是当前估计$v_t(s_t)$ 与TD 误差的综合结果。

状态值的TD 学习——算法性质

TD 算法可标注为：

$$\underbrace{v_{t+1}(s_t)}_{\text{新估计}} = \underbrace{v_t(s_t)}_{\text{当前估计}} - \alpha_t(s_t)\big[\overbrace{v_t(s_t) - \underbrace{[r_{t+1} + \gamma v_t(s_{t+1})]}_{\text{TD 目标}\bar{v}_t}}^{\text{TD 误差}\delta_t}\big], \tag{3}$$

其中，

$$\bar{v}_t \doteq r_{t+1} + \gamma v_t(s_{t+1})$$

称为TD 目标。

$$\delta_t \doteq v_t(s_t) - [r_{t+1} + \gamma v_t(s_{t+1})] = v_t(s_t) - \bar{v}_t$$

称为TD 误差。

观察：新估计$v_{t+1}(s_t)$ 是当前估计$v_t(s_t)$ 与TD 误差的综合结果。

状态值的TD 学习——算法性质

TD 算法可标注为：

$$\underbrace{v_{t+1}(s_t)}_{\text{新估计}} = \underbrace{v_t(s_t)}_{\text{当前估计}} - \alpha_t(s_t)\big[\overbrace{v_t(s_t) - \underbrace{[r_{t+1} + \gamma v_t(s_{t+1})]}_{\text{TD 目标}\bar{v}_t}}^{\text{TD 误差}\delta_t}\big], \tag{3}$$

其中，

$$\bar{v}_t \doteq r_{t+1} + \gamma v_t(s_{t+1})$$

称为TD 目标。

$$\delta_t \doteq v_t(s_t) - [r_{t+1} + \gamma v_t(s_{t+1})] = v_t(s_t) - \bar{v}_t$$

称为TD 误差。

观察：新估计$v_{t+1}(s_t)$ 是当前估计$v_t(s_t)$ 与TD 误差的综合结果。

状态值的TD 学习——算法性质

TD 算法可标注为：

$$\underbrace{v_{t+1}(s_t)}_{\text{新估计}} = \underbrace{v_t(s_t)}_{\text{当前估计}} - \alpha_t(s_t)\big[\overbrace{v_t(s_t) - \underbrace{[r_{t+1} + \gamma v_t(s_{t+1})]}_{\text{TD 目标}\bar{v}_t}}^{\text{TD 误差}\delta_t}\big], \tag{3}$$

其中，

$$\bar{v}_t \doteq r_{t+1} + \gamma v_t(s_{t+1})$$

称为TD 目标。

$$\delta_t \doteq v_t(s_t) - [r_{t+1} + \gamma v_t(s_{t+1})] = v_t(s_t) - \bar{v}_t$$

称为TD 误差。

观察：新估计$v_{t+1}(s_t)$ 是当前估计$v_t(s_t)$ 与TD 误差的综合结果。

状态值的TD 学习——算法性质

第一，为什么$\bar{v}_t$ 称为TD 目标?

因为该算法驱动$v(s_t)$ 向$\bar{v}_t$ 靠近。

证明如下:

$$v_{t+1}(s_t) = v_t(s_t) - \alpha_t(s_t)\big[v_t(s_t) - \bar{v}_t\big]$$

状态值的TD 学习——算法性质

第一，为什么$\bar{v}_t$ 称为TD 目标?

因为该算法驱动$v(s_t)$ 向$\bar{v}_t$ 靠近。

证明如下:

$$v_{t+1}(s_t) = v_t(s_t) - \alpha_t(s_t)\big[v_t(s_t) - \bar{v}_t\big]$$

状态值的TD 学习——算法性质

第一，为什么$\bar{v}_t$ 称为TD 目标?

因为该算法驱动$v(s_t)$ 向$\bar{v}_t$ 靠近。

证明如下：

$$v_{t+1}(s_t) = v_t(s_t) - \alpha_t(s_t)\big[v_t(s_t) - \bar{v}_t\big]$$

状态值的TD 学习——算法性质

第一，为什么$\bar{v}_t$ 称为TD 目标?

因为该算法驱动$v(s_t)$ 向$\bar{v}_t$ 靠近。

证明如下：

$$
\begin{aligned}
& v_{t+1}(s_t) = v_t(s_t) - \alpha_t(s_t)\big[v_t(s_t) - \bar{v}_t\big] \\
\Longrightarrow \quad & v_{t+1}(s_t) - \bar{v}_t = v_t(s_t) - \bar{v}_t - \alpha_t(s_t)\big[v_t(s_t) - \bar{v}_t\big]
\end{aligned}
$$

状态值的TD 学习——算法性质

第一，为什么$\bar{v}_t$ 称为TD 目标?

因为该算法驱动$v(s_t)$ 向$\bar{v}_t$ 靠近。

证明如下：

$$
\begin{aligned}
& v_{t+1}(s_t) = v_t(s_t) - \alpha_t(s_t)\big[v_t(s_t) - \bar{v}_t\big] \\
\Longrightarrow \quad & v_{t+1}(s_t) - \bar{v}_t = v_t(s_t) - \bar{v}_t - \alpha_t(s_t)\big[v_t(s_t) - \bar{v}_t\big] \\
\Longrightarrow \quad & v_{t+1}(s_t) - \bar{v}_t = [1 - \alpha_t(s_t)][v_t(s_t) - \bar{v}_t]
\end{aligned}
$$

状态值的TD 学习——算法性质

第一，为什么$\bar{v}_t$ 称为TD 目标?

因为该算法驱动$v(s_t)$ 向$\bar{v}_t$ 靠近。

证明如下：

$$
\begin{aligned}
& v_{t+1}(s_t) = v_t(s_t) - \alpha_t(s_t)\big[v_t(s_t) - \bar{v}_t\big] \\
\Longrightarrow \quad & v_{t+1}(s_t) - \bar{v}_t = v_t(s_t) - \bar{v}_t - \alpha_t(s_t)\big[v_t(s_t) - \bar{v}_t\big] \\
\Longrightarrow \quad & v_{t+1}(s_t) - \bar{v}_t = [1 - \alpha_t(s_t)][v_t(s_t) - \bar{v}_t] \\
\Longrightarrow \quad & |v_{t+1}(s_t) - \bar{v}_t| = |1 - \alpha_t(s_t)||v_t(s_t) - \bar{v}_t|
\end{aligned}
$$

状态值的TD 学习——算法性质

第一，为什么$\bar{v}_t$ 称为TD 目标?

因为该算法驱动$v(s_t)$ 向$\bar{v}_t$ 靠近。

证明如下:

$$
\begin{aligned}
& v_{t+1}(s_t) = v_t(s_t) - \alpha_t(s_t)\big[v_t(s_t) - \bar{v}_t\big] \\
\Longrightarrow \quad & v_{t+1}(s_t) - \bar{v}_t = v_t(s_t) - \bar{v}_t - \alpha_t(s_t)\big[v_t(s_t) - \bar{v}_t\big] \\
\Longrightarrow \quad & v_{t+1}(s_t) - \bar{v}_t = [1 - \alpha_t(s_t)][v_t(s_t) - \bar{v}_t] \\
\Longrightarrow \quad & |v_{t+1}(s_t) - \bar{v}_t| = |1 - \alpha_t(s_t)||v_t(s_t) - \bar{v}_t|
\end{aligned}
$$

由于$\alpha_t(s_t)$ 是一个小正数，因此:

$$0 < 1 - \alpha_t(s_t) < 1$$

状态值的TD 学习——算法性质

第一，为什么$\bar{v}_t$ 称为TD 目标?

因为该算法驱动$v(s_t)$ 向$\bar{v}_t$ 靠近。

证明如下：

$$
\begin{aligned}
& v_{t+1}(s_t) = v_t(s_t) - \alpha_t(s_t)\big[v_t(s_t) - \bar{v}_t\big] \\
\Longrightarrow \quad & v_{t+1}(s_t) - \bar{v}_t = v_t(s_t) - \bar{v}_t - \alpha_t(s_t)\big[v_t(s_t) - \bar{v}_t\big] \\
\Longrightarrow \quad & v_{t+1}(s_t) - \bar{v}_t = [1 - \alpha_t(s_t)][v_t(s_t) - \bar{v}_t] \\
\Longrightarrow \quad & |v_{t+1}(s_t) - \bar{v}_t| = |1 - \alpha_t(s_t)||v_t(s_t) - \bar{v}_t|
\end{aligned}
$$

由于$\alpha_t(s_t)$ 是一个小正数，因此：

$$0 < 1 - \alpha_t(s_t) < 1$$

从而：

$$|v_{t+1}(s_t) - \bar{v}_t| \leq |v_t(s_t) - \bar{v}_t|$$

这说明$v(s_t)$ 被驱动向$\bar{v}_t$ 靠近！

第二，TD 误差的含义是什么?

$$\delta_t = v_t(s_t) - [r_{t+1} + \gamma v_t(s_{t+1})]$$

- 它反映了两个时刻之间的差异。
- 它反映了v_t 与v_π 之间的差异。为此，定义:

$$\delta_{\pi,t} \doteq v_\pi(s_t) - [r_{t+1} + \gamma v_\pi(s_{t+1})]$$

注意到:

$$\mathbb{E}[\delta_{\pi,t}|S_t = s_t] = v_\pi(s_t) - \mathbb{E}\big[R_{t+1} + \gamma v_\pi(S_{t+1})|S_t = s_t\big] = 0.$$

- 若$v_t = v_\pi$，则δ_t 在期望意义下应为零。
- 因此，若δ_t 不为零，则v_t 与v_π 不相等。

- TD 误差可以理解为新息（innovation），即从经验(s_t, r_{t+1}, s_{t+1}) 中获得的新信息。

第二，TD 误差的含义是什么?

$$\delta_t = v_t(s_t) - [r_{t+1} + \gamma v_t(s_{t+1})]$$

- 它反映了两个时刻之间的差异。
- 它反映了v_t 与v_π 之间的差异。为此，定义:

$$\delta_{\pi,t} \doteq v_\pi(s_t) - [r_{t+1} + \gamma v_\pi(s_{t+1})]$$

注意到:

$$\mathbb{E}[\delta_{\pi,t}|S_t = s_t] = v_\pi(s_t) - \mathbb{E}\big[R_{t+1} + \gamma v_\pi(S_{t+1})|S_t = s_t\big] = 0.$$

- 若$v_t = v_\pi$，则δ_t 在期望意义下应为零。
- 因此，若δ_t 不为零，则v_t 与v_π 不相等。

- TD 误差可以理解为新息（innovation），即从经验(s_t, r_{t+1}, s_{t+1}) 中获得的新信息。

第二，TD 误差的含义是什么?

$$\delta_t = v_t(s_t) - [r_{t+1} + \gamma v_t(s_{t+1})]$$

- 它反映了两个时刻之间的差异。
- 它反映了v_t 与v_π 之间的差异。为此，定义:

$$\delta_{\pi,t} \doteq v_\pi(s_t) - [r_{t+1} + \gamma v_\pi(s_{t+1})]$$

注意到:

$$\mathbb{E}[\delta_{\pi,t}|S_t = s_t] = v_\pi(s_t) - \mathbb{E}\big[R_{t+1} + \gamma v_\pi(S_{t+1})|S_t = s_t\big] = 0.$$

- 若$v_t = v_\pi$，则δ_t 在期望意义下应为零。
- 因此，若δ_t 不为零，则v_t 与v_π 不相等。

- TD 误差可以理解为新息（innovation），即从经验(s_t, r_{t+1}, s_{t+1}) 中获得的新信息。

第二，TD 误差的含义是什么?

$$\delta_t = v_t(s_t) - [r_{t+1} + \gamma v_t(s_{t+1})]$$

- 它反映了两个时刻之间的差异。
- 它反映了v_t 与v_π 之间的差异。为此，定义:

$$\delta_{\pi,t} \doteq v_\pi(s_t) - [r_{t+1} + \gamma v_\pi(s_{t+1})]$$

注意到:

$$\mathbb{E}[\delta_{\pi,t}|S_t = s_t] = v_\pi(s_t) - \mathbb{E}\big[R_{t+1} + \gamma v_\pi(S_{t+1})|S_t = s_t\big] = 0.$$

- 若$v_t = v_\pi$，则δ_t 在期望意义下应为零。
- 因此，若δ_t 不为零，则v_t 与v_π 不相等。

- TD 误差可以理解为新息（innovation），即从经验(s_t, r_{t+1}, s_{t+1}) 中获得的新信息。

第二，TD 误差的含义是什么?

$$\delta_t = v_t(s_t) - [r_{t+1} + \gamma v_t(s_{t+1})]$$

- 它反映了两个时刻之间的差异。
- 它反映了v_t 与v_π 之间的差异。为此，定义:

$$\delta_{\pi,t} \doteq v_\pi(s_t) - [r_{t+1} + \gamma v_\pi(s_{t+1})]$$

注意到:

$$\mathbb{E}[\delta_{\pi,t}|S_t = s_t] = v_\pi(s_t) - \mathbb{E}\big[R_{t+1} + \gamma v_\pi(S_{t+1})|S_t = s_t\big] = 0.$$

- 若$v_t = v_\pi$，则δ_t 在期望意义下应为零。
- 因此，若δ_t 不为零，则v_t 与v_π 不相等。

- TD 误差可以理解为新息（innovation），即从经验(s_t, r_{t+1}, s_{t+1}) 中获得的新信息。

第二，TD 误差的含义是什么?

$$\delta_t = v_t(s_t) - [r_{t+1} + \gamma v_t(s_{t+1})]$$

- 它反映了两个时刻之间的差异。
- 它反映了v_t 与v_π 之间的差异。为此，定义:

$$\delta_{\pi,t} \doteq v_\pi(s_t) - [r_{t+1} + \gamma v_\pi(s_{t+1})]$$

注意到:

$$\mathbb{E}[\delta_{\pi,t}|S_t = s_t] = v_\pi(s_t) - \mathbb{E}\big[R_{t+1} + \gamma v_\pi(S_{t+1})|S_t = s_t\big] = 0.$$

- 若$v_t = v_\pi$，则δ_t 在期望意义下应为零。
- 因此，若δ_t 不为零，则v_t 与v_π 不相等。

- TD 误差可以理解为新息（innovation），即从经验(s_t, r_{t+1}, s_{t+1}) 中获得的新信息。

第二，TD 误差的含义是什么?

$$\delta_t = v_t(s_t) - [r_{t+1} + \gamma v_t(s_{t+1})]$$

- 它反映了两个时刻之间的差异。
- 它反映了v_t 与v_π 之间的差异。为此，定义:

$$\delta_{\pi,t} \doteq v_\pi(s_t) - [r_{t+1} + \gamma v_\pi(s_{t+1})]$$

注意到:

$$\mathbb{E}[\delta_{\pi,t}|S_t = s_t] = v_\pi(s_t) - \mathbb{E}\big[R_{t+1} + \gamma v_\pi(S_{t+1})|S_t = s_t\big] = 0.$$

- 若$v_t = v_\pi$，则δ_t 在期望意义下应为零。
- 因此，若δ_t 不为零，则v_t 与v_π 不相等。

- TD 误差可以理解为新息（innovation），即从经验(s_t, r_{t+1}, s_{t+1}) 中获得的新信息。

状态值的TD 学习——算法性质

其他性质：

- 式(3) 中的TD 算法只估计给定策略的状态值。
  - 它不估计动作值。
  - 它不搜索最优策略。
- 该算法将在本讲后续内容中被扩展，用于估计动作值并搜索最优策略。
- 式(3) 中的TD 算法是理解更复杂TD 算法的基础。

状态值的TD 学习——算法思想

问：TD 算法在数学上做了什么？

答：它是求解给定策略π 的Bellman 方程的无模型算法。

- 第2 章介绍了求解Bellman 方程的基于模型的算法：闭合解+ 迭代算法。

状态值的TD 学习——算法思想

第一，Bellman 方程的新表达形式。

策略π 的状态值定义为:

$$v_\pi(s) = \mathbb{E}\big[R + \gamma G | S = s\big], \quad s \in \mathcal{S} \tag{4}$$

其中G 为折扣回报。由于:

$$\mathbb{E}[G|S = s] = \sum_a \pi(a|s) \sum_{s'} p(s'|s, a) v_\pi(s') = \mathbb{E}[v_\pi(S')|S = s],$$

其中S' 为下一状态，可将式(4) 改写为:

$$v_\pi(s) = \mathbb{E}\big[R + \gamma v_\pi(S') | S = s\big], \quad s \in \mathcal{S}. \tag{5}$$

式(5) 是Bellman 方程的另一种表达形式，有时称为Bellman 期望方程，是设计和分析TD 算法的重要工具。

状态值的TD 学习——算法思想

第一，Bellman 方程的新表达形式。

策略π 的状态值定义为:

$$v_\pi(s) = \mathbb{E}\big[R + \gamma G | S = s\big], \quad s \in \mathcal{S} \tag{4}$$

其中G 为折扣回报。由于:

$$\mathbb{E}[G|S=s] = \sum_a \pi(a|s) \sum_{s'} p(s'|s,a) v_\pi(s') = \mathbb{E}[v_\pi(S')|S=s],$$

其中S' 为下一状态，可将式(4) 改写为:

$$v_\pi(s) = \mathbb{E}\big[R + \gamma v_\pi(S') | S = s\big], \quad s \in \mathcal{S}. \tag{5}$$

式(5) 是Bellman 方程的另一种表达形式，有时称为Bellman 期望方程，是设计和分析TD 算法的重要工具。

状态值的TD 学习——算法思想

第一，Bellman 方程的新表达形式。

策略π 的状态值定义为:

$$v_\pi(s) = \mathbb{E}\big[R + \gamma G | S = s\big], \quad s \in \mathcal{S} \tag{4}$$

其中G 为折扣回报。由于:

$$\mathbb{E}[G|S=s] = \sum_a \pi(a|s) \sum_{s'} p(s'|s,a) v_\pi(s') = \mathbb{E}[v_\pi(S')|S=s],$$

其中S' 为下一状态，可将式(4) 改写为:

$$v_\pi(s) = \mathbb{E}\big[R + \gamma v_\pi(S') | S = s\big], \quad s \in \mathcal{S}. \tag{5}$$

式(5) 是Bellman 方程的另一种表达形式，有时称为Bellman 期望方程，是设计和分析TD 算法的重要工具。

第一，Bellman 方程的新表达形式。

策略π 的状态值定义为：

$$v_\pi(s) = \mathbb{E}\big[R + \gamma G | S = s\big], \quad s \in \mathcal{S} \tag{4}$$

其中G 为折扣回报。由于：

$$\mathbb{E}[G|S=s] = \sum_a \pi(a|s) \sum_{s'} p(s'|s,a) v_\pi(s') = \mathbb{E}[v_\pi(S')|S=s],$$

其中S' 为下一状态，可将式(4) 改写为：

$$v_\pi(s) = \mathbb{E}\big[R + \gamma v_\pi(S') | S = s\big], \quad s \in \mathcal{S}. \tag{5}$$

式(5) 是Bellman 方程的另一种表达形式，有时称为Bellman 期望方程，是设计和分析TD 算法的重要工具。

第一，Bellman 方程的新表达形式。

策略π 的状态值定义为：

$$v_\pi(s) = \mathbb{E}\big[R + \gamma G | S = s\big], \quad s \in \mathcal{S} \tag{4}$$

其中G 为折扣回报。由于：

$$\mathbb{E}[G|S=s] = \sum_a \pi(a|s) \sum_{s'} p(s'|s,a) v_\pi(s') = \mathbb{E}[v_\pi(S')|S=s],$$

其中S' 为下一状态，可将式(4) 改写为：

$$v_\pi(s) = \mathbb{E}\big[R + \gamma v_\pi(S') | S = s\big], \quad s \in \mathcal{S}. \tag{5}$$

式(5) 是Bellman 方程的另一种表达形式，有时称为Bellman 期望方程，是设计和分析TD 算法的重要工具。

状态值的TD 学习——算法思想

第二，使用RM 算法求解式(5) 中的Bellman 方程。
具体地，定义：

$$g(v(s)) = v(s) - \mathbb{E}\big[R + \gamma v_\pi(S')|s\big],$$

可将式(5) 改写为：

$$g(v(s)) = 0.$$

由于我们只能获得R 和S' 的样本r 和s'，带噪声的观测为：

$$\begin{aligned}\tilde{g}(v(s)) &= v(s) - \big[r + \gamma v_\pi(s')\big] \\ &= \underbrace{\Big(v(s) - \mathbb{E}\big[R + \gamma v_\pi(S')|s\big]\Big)}_{g(v(s))} + \underbrace{\Big(\mathbb{E}\big[R + \gamma v_\pi(S')|s\big] - \big[r + \gamma v_\pi(s')\big]\Big)}_{\eta}.\end{aligned}$$

第二，使用RM 算法求解式(5) 中的Bellman 方程。
具体地，定义：

$$g(v(s)) = v(s) - \mathbb{E}\big[R + \gamma v_\pi(S')|s\big],$$

可将式(5) 改写为：

$$g(v(s)) = 0.$$

由于我们只能获得R 和S' 的样本r 和s'，带噪声的观测为：

$$\begin{aligned}\tilde{g}(v(s)) &= v(s) - \big[r + \gamma v_\pi(s')\big] \\ &= \underbrace{\Big(v(s) - \mathbb{E}\big[R + \gamma v_\pi(S')|s\big]\Big)}_{g(v(s))} + \underbrace{\Big(\mathbb{E}\big[R + \gamma v_\pi(S')|s\big] - \big[r + \gamma v_\pi(s')\big]\Big)}_{\eta}.\end{aligned}$$

状态值的TD 学习——算法思想

因此，求解$g(v(s)) = 0$ 的RM 算法为：

$$
\begin{aligned}
v_{k+1}(s) &= v_k(s) - \alpha_k \tilde{g}(v_k(s)) \\
&= v_k(s) - \alpha_k \Big(v_k(s) - \big[r_k + \gamma v_\pi(s'_k) \big] \Big), \quad k = 1, 2, 3, \ldots
\end{aligned} \tag{6}
$$

其中$v_k(s)$ 是第k 步对$v_\pi(s)$ 的估计；r_k, s'_k 是第k 步获得的R, S' 的样本。

式(6) 中的RM 算法 与TD 算法 非常相似，但存在**两个区别**：

- 区别一：RM 算法需要$\{(s, r_k, s'_k)\}$，$k = 1, 2, 3, \ldots$。
  - 修改：将$\{(s, r_k, s'_k)\}$ 改为$\{(s_t, r_{t+1}, s_{t+1})\}$，使算法能利用一个回合中的时序样本。
- 区别二：RM 算法需要$v_\pi(s'_k)$。
  - 修改：将$v_\pi(s'_k)$ 替换为估计值$v_t(s_{t+1})$。

经过以上修改，RM 算法恰好变为TD 算法。

状态值的TD 学习——算法思想

因此，求解$g(v(s)) = 0$ 的RM 算法为：

$$
\begin{aligned}
v_{k+1}(s) &= v_k(s) - \alpha_k \tilde{g}(v_k(s)) \\
&= v_k(s) - \alpha_k \Big(v_k(s) - \big[r_k + \gamma v_\pi(s'_k) \big] \Big), \quad k = 1, 2, 3, \ldots
\end{aligned} \tag{6}
$$

其中$v_k(s)$ 是第k 步对$v_\pi(s)$ 的估计；r_k, s'_k 是第k 步获得的R, S' 的样本。

式(6) 中的RM 算法 与TD 算法 非常相似，但存在**两个区别**：

- 区别一：RM 算法需要$\{(s, r_k, s'_k)\}$，$k = 1, 2, 3, \ldots$。
  - 修改：将$\{(s, r_k, s'_k)\}$ 改为$\{(s_t, r_{t+1}, s_{t+1})\}$，使算法能利用一个回合中的时序样本。
- 区别二：RM 算法需要$v_\pi(s'_k)$。
  - 修改：将$v_\pi(s'_k)$ 替换为估计值$v_t(s_{t+1})$。

经过以上修改，RM 算法恰好变为TD 算法。

状态值的TD 学习——算法思想

因此，求解$g(v(s)) = 0$ 的RM 算法为：

$$
\begin{aligned}
v_{k+1}(s) &= v_k(s) - \alpha_k \tilde{g}(v_k(s)) \\
&= v_k(s) - \alpha_k \Big(v_k(s) - \big[r_k + \gamma v_\pi(s'_k) \big] \Big), \quad k = 1, 2, 3, \ldots
\end{aligned} \tag{6}
$$

其中$v_k(s)$ 是第k 步对$v_\pi(s)$ 的估计；r_k, s'_k 是第k 步获得的R, S' 的样本。

式(6) 中的RM 算法 与TD 算法 非常相似，但存在**两个区别**：

- 区别一：RM 算法需要$\{(s, r_k, s'_k)\}$，$k = 1, 2, 3, \ldots$。
  - 修改：将$\{(s, r_k, s'_k)\}$ 改为$\{(s_t, r_{t+1}, s_{t+1})\}$，使算法能利用一个回合中的时序样本。
- 区别二：RM 算法需要$v_\pi(s'_k)$。
  - 修改：将$v_\pi(s'_k)$ 替换为估计值$v_t(s_{t+1})$。

经过以上修改，RM 算法恰好变为TD 算法。

状态值的TD 学习——算法思想

因此，求解$g(v(s)) = 0$ 的RM 算法为：

$$
\begin{aligned}
v_{k+1}(s) &= v_k(s) - \alpha_k \tilde{g}(v_k(s)) \\
&= v_k(s) - \alpha_k \Big(v_k(s) - \big[r_k + \gamma v_\pi(s'_k) \big] \Big), \quad k = 1, 2, 3, \ldots
\end{aligned} \tag{6}
$$

其中$v_k(s)$ 是第k 步对$v_\pi(s)$ 的估计；r_k, s'_k 是第k 步获得的R, S' 的样本。

式(6) 中的RM 算法 与TD 算法 非常相似，但存在**两个区别**：

- 区别一：RM 算法需要$\{(s, r_k, s'_k)\}$，$k = 1, 2, 3, \ldots$。
  - 修改：将$\{(s, r_k, s'_k)\}$ 改为$\{(s_t, r_{t+1}, s_{t+1})\}$，使算法能利用一个回合中的时序样本。
- 区别二：RM 算法需要$v_\pi(s'_k)$。
  - 修改：将$v_\pi(s'_k)$ 替换为估计值$v_t(s_{t+1})$。

经过以上修改，RM 算法恰好变为TD 算法。

状态值的TD 学习——算法思想

因此，求解$g(v(s)) = 0$ 的RM 算法为：

$$
\begin{aligned}
v_{k+1}(s) &= v_k(s) - \alpha_k \tilde{g}(v_k(s)) \\
&= v_k(s) - \alpha_k \Big(v_k(s) - \big[r_k + \gamma v_\pi(s'_k) \big] \Big), \quad k = 1, 2, 3, \ldots
\end{aligned} \tag{6}
$$

其中$v_k(s)$ 是第k 步对$v_\pi(s)$ 的估计；r_k, s'_k 是第k 步获得的R, S' 的样本。

式(6) 中的RM 算法 与TD 算法 非常相似，但存在**两个区别**：

- 区别一：RM 算法需要$\{(s, r_k, s'_k)\}$，$k = 1, 2, 3, \ldots$。
  - 修改：将$\{(s, r_k, s'_k)\}$ 改为$\{(s_t, r_{t+1}, s_{t+1})\}$，使算法能利用一个回合中的时序样本。
- 区别二：RM 算法需要$v_\pi(s'_k)$。
  - 修改：将$v_\pi(s'_k)$ 替换为估计值$v_t(s_{t+1})$。

经过以上修改，RM 算法恰好变为TD 算法。

状态值的TD 学习——算法收敛性

Theorem (TD 学习的收敛性)

由*TD* 算法(1)，若对所有$s \in \mathcal{S}$ 满足$\sum_t \alpha_t(s) = \infty$ 且$\sum_t \alpha_t^2(s) < \infty$，则$v_t(s)$ 以概率*1* 收敛到$v_\pi(s)$（$t \to \infty$）。

定理的证明可参见教材。

注：

- 该定理指出，对于给定策略π，TD 算法可以求得状态值。
- $\sum_t \alpha_t(s) = \infty$ 且$\sum_t \alpha_t^2(s) < \infty$ 必须对所有$s \in \mathcal{S}$ 成立。
  - 对于条件$\sum_t \alpha_t(s) = \infty$：在时刻$t$，
    - ◇ 若$s = s_t$，则$\alpha_t(s) > 0$；
    - ◇ 若$s \neq s_t$，则$\alpha_t(s) = 0$。

    因此，$\sum_t \alpha_t(s) = \infty$ 要求每个状态必须被访问无穷多次（或足够多次）。
  - 对于条件$\sum_t \alpha_t^2(s) < \infty$：实践中学习率$\alpha$ 通常取小的常数，此时该条件不再满足。但当α 为常数时，仍可证明算法在期望意义下收敛。

状态值的TD 学习——算法收敛性

Theorem (TD 学习的收敛性)

由*TD* 算法(1)，若对所有$s \in \mathcal{S}$ 满足$\sum_t \alpha_t(s) = \infty$ 且$\sum_t \alpha_t^2(s) < \infty$，则$v_t(s)$ 以概率*1* 收敛到$v_\pi(s)$（$t \to \infty$）。

定理的证明可参见教材。

注：

- 该定理指出，对于给定策略π，TD 算法可以求得状态值。
- $\sum_t \alpha_t(s) = \infty$ 且$\sum_t \alpha_t^2(s) < \infty$ 必须对所有$s \in \mathcal{S}$ 成立。
  - 对于条件$\sum_t \alpha_t(s) = \infty$：在时刻$t$，
    - 若$s = s_t$，则$\alpha_t(s) > 0$；
    - 若$s \neq s_t$，则$\alpha_t(s) = 0$。

    因此，$\sum_t \alpha_t(s) = \infty$ 要求每个状态必须被访问无穷多次（或足够多次）。
  - 对于条件$\sum_t \alpha_t^2(s) < \infty$：实践中学习率$\alpha$ 通常取小的常数，此时该条件不再满足。但当α 为常数时，仍可证明算法在期望意义下收敛。

状态值的TD 学习——算法收敛性

Theorem (TD 学习的收敛性)

由*TD* 算法(1)，若对所有$s \in \mathcal{S}$ 满足$\sum_t \alpha_t(s) = \infty$ 且$\sum_t \alpha_t^2(s) < \infty$，则$v_t(s)$ 以概率*1* 收敛到$v_\pi(s)$（$t \to \infty$）。

定理的证明可参见教材。

注:

- 该定理指出，对于给定策略π，TD 算法可以求得状态值。
- $\sum_t \alpha_t(s) = \infty$ 且$\sum_t \alpha_t^2(s) < \infty$ 必须对所有$s \in \mathcal{S}$ 成立。
  - 对于条件$\sum_t \alpha_t(s) = \infty$：在时刻$t$，
    - ◇ 若$s = s_t$，则$\alpha_t(s) > 0$;
    - ◇ 若$s \neq s_t$，则$\alpha_t(s) = 0$。

    因此，$\sum_t \alpha_t(s) = \infty$ 要求每个状态必须被访问无穷多次（或足够多次）。
  - 对于条件$\sum_t \alpha_t^2(s) < \infty$：实践中学习率$\alpha$ 通常取小的常数，此时该条件不再满足。但当α 为常数时，仍可证明算法在期望意义下收敛。

状态值的TD 学习——算法收敛性

Theorem (TD 学习的收敛性)

由*TD* 算法(1)，若对所有$s \in \mathcal{S}$ 满足$\sum_t \alpha_t(s) = \infty$ 且$\sum_t \alpha_t^2(s) < \infty$，则$v_t(s)$ 以概率*1* 收敛到$v_\pi(s)$（$t \to \infty$）。

定理的证明可参见教材。

注:

- 该定理指出，对于给定策略π，TD 算法可以求得状态值。
- $\sum_t \alpha_t(s) = \infty$ 且$\sum_t \alpha_t^2(s) < \infty$ 必须对所有$s \in \mathcal{S}$ 成立。
  - 对于条件$\sum_t \alpha_t(s) = \infty$：在时刻$t$，
    - ◇ 若$s = s_t$，则$\alpha_t(s) > 0$;
    - ◇ 若$s \neq s_t$，则$\alpha_t(s) = 0$。

    因此，$\sum_t \alpha_t(s) = \infty$ 要求每个状态必须被访问无穷多次（或足够多次）。
  - 对于条件$\sum_t \alpha_t^2(s) < \infty$：实践中学习率$\alpha$ 通常取小的常数，此时该条件不再满足。但当α 为常数时，仍可证明算法在期望意义下收敛。

状态值的TD 学习——算法收敛性

Theorem (TD 学习的收敛性)

由*TD* 算法(1)，若对所有$s \in \mathcal{S}$ 满足$\sum_t \alpha_t(s) = \infty$ 且$\sum_t \alpha_t^2(s) < \infty$，则$v_t(s)$ 以概率*1* 收敛到$v_\pi(s)$（$t \to \infty$）。

定理的证明可参见教材。

注:

- 该定理指出，对于给定策略π，TD 算法可以求得状态值。
- $\sum_t \alpha_t(s) = \infty$ 且$\sum_t \alpha_t^2(s) < \infty$ 必须对所有$s \in \mathcal{S}$ 成立。
  - 对于条件$\sum_t \alpha_t(s) = \infty$：在时刻$t$，
    - ◇ 若$s = s_t$，则$\alpha_t(s) > 0$;
    - ◇ 若$s \neq s_t$，则$\alpha_t(s) = 0$。

    因此，$\sum_t \alpha_t(s) = \infty$ 要求每个状态必须被访问无穷多次（或足够多次）。
  - 对于条件$\sum_t \alpha_t^2(s) < \infty$：实践中学习率$\alpha$ 通常取小的常数，此时该条件不再满足。但当α 为常数时，仍可证明算法在期望意义下收敛。

状态值的TD 学习——算法性质

TD 学习与MC 学习同为无模型方法，那么TD 学习相比MC 学习有哪些**优缺点**?

TD / Sarsa 学习	MC 学习

Table: TD 学习与MC 学习的比较。

状态值的TD 学习——算法性质

TD 学习与MC 学习同为无模型方法，那么TD 学习相比MC 学习有哪些**优缺点**?

TD / Sarsa 学习	MC 学习

Table: TD 学习与MC 学习的比较。

状态值的TD 学习——算法性质

TD 学习与MC 学习同为无模型方法，那么TD 学习相比MC 学习有哪些**优缺点**?

TD / Sarsa 学习	MC 学习
在线（Online）： TD 学习是在线的，每收到一个奖励后即可立即更新状态/动作值。	

Table: TD 学习与MC 学习的比较。

状态值的TD 学习——算法性质

TD 学习与MC 学习同为无模型方法，那么TD 学习相比MC 学习有哪些**优缺点**?

TD / Sarsa 学习	MC 学习
在线（Online）： TD 学习是在线的，每收到一个奖励后即可立即更新状态/动作值。	**离线（Offline）**： MC 学习是离线的，必须等待整个回合采集完毕后才能更新。

Table: TD 学习与MC 学习的比较。

状态值的TD 学习——算法性质

TD 学习与MC 学习同为无模型方法，那么TD 学习相比MC 学习有哪些**优缺点**?

TD / Sarsa 学习	MC 学习
在线（Online）： TD 学习是在线的，每收到一个奖励后即可立即更新状态/动作值。	**离线（Offline）**： MC 学习是离线的，必须等待整个回合采集完毕后才能更新。
持续性任务： 由于TD 学习是在线的，它既能处理回合制任务，也能处理持续性任务。	

Table: TD 学习与MC 学习的比较。

状态值的TD 学习——算法性质

TD 学习与MC 学习同为无模型方法，那么TD 学习相比MC 学习有哪些**优缺点**?

TD / Sarsa 学习	MC 学习
在线（Online）： TD 学习是在线的，每收到一个奖励后即可立即更新状态/动作值。	**离线（Offline）**： MC 学习是离线的，必须等待整个回合采集完毕后才能更新。
持续性任务： 由于TD 学习是在线的，它既能处理回合制任务，也能处理持续性任务。	**回合制任务**： 由于MC 学习是离线的，它只能处理具有终止状态的回合制任务。

Table: TD 学习与MC 学习的比较。

状态值的TD 学习——算法性质

TD 学习与MC 学习同为无模型方法，那么TD 学习相比MC 学习有哪些**优缺点**?

TD / Sarsa 学习	MC 学习

Table: TD 学习与MC 学习的比较（续）。

状态值的TD 学习——算法性质

TD 学习与MC 学习同为无模型方法，那么TD 学习相比MC 学习有哪些**优缺点**?

TD / Sarsa 学习	MC 学习
自举（Bootstrapping）： TD 是自举的，因为值的更新依赖于该值之前的估计，因此需要初始猜测值。	

Table: TD 学习与MC 学习的比较（续）。

状态值的TD 学习——算法性质

TD 学习与MC 学习同为无模型方法，那么TD 学习相比MC 学习有哪些**优缺点**?

TD / Sarsa 学习	MC 学习
自举（Bootstrapping）： TD 是自举的，因为值的更新依赖于该值之前的估计，因此需要初始猜测值。	**非自举**： MC 不是自举的，它可以在无需任何初始猜测的情况下直接估计状态/动作值。

Table: TD 学习与MC 学习的比较（续）。

状态值的TD 学习——算法性质

TD 学习与MC 学习同为无模型方法，那么TD 学习相比MC 学习有哪些**优缺点**?

TD / Sarsa 学习	MC 学习
自举（Bootstrapping）： TD 是自举的，因为值的更新依赖于该值之前的估计，因此需要初始猜测值。	**非自举**： MC 不是自举的，它可以在无需任何初始猜测的情况下直接估计状态/动作值。
低估计方差： TD 的方差低于MC，因为涉及的随机变量更少。例如，Sarsa 仅需要$R_{t+1}, S_{t+1}, A_{t+1}$。	

Table: TD 学习与MC 学习的比较（续）。

状态值的TD 学习——算法性质

TD 学习与MC 学习同为无模型方法，那么TD 学习相比MC 学习有哪些**优缺点**?

TD / Sarsa 学习	MC 学习
自举（Bootstrapping）： TD 是自举的，因为值的更新依赖于该值之前的估计，因此需要初始猜测值。	**非自举**： MC 不是自举的，它可以在无需任何初始猜测的情况下直接估计状态/动作值。
低估计方差： TD 的方差低于MC，因为涉及的随机变量更少。例如，Sarsa 仅需要$R_{t+1}, S_{t+1}, A_{t+1}$。	**高估计方差**： 估计$q_\pi(s_t, a_t)$ 需要$R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \ldots$ 的样本。若每个回合长度为L，则共有$\lvert\mathcal{A}\rvert^L$种可能的回合。

Table: TD 学习与MC 学习的比较（续）。

大纲

1 动机示例

2 状态值的TD 学习

3 动作值的TD 学习：Sarsa

4 动作值的TD 学习：n 步Sarsa

5 最优动作值的TD 学习：Q 学习

6 统一视角

7 总结

动作值的TD 学习——Sarsa

- 上一节介绍的TD 算法只能估计状态值。
- 接下来，我们介绍Sarsa 算法，它可以直接估计动作值。
- 我们还将看到如何用Sarsa 寻找最优策略。

Sarsa——算法

首先，我们的目标是估计给定策略π 的动作值。

假设我们有一些经验$\{(s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1})\}_t$。
可以使用如下*Sarsa* 算法估计动作值:

$$q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha_t(s_t, a_t)\Big[q_t(s_t, a_t) - [r_{t+1} + \gamma q_t(s_{t+1}, a_{t+1})]\Big],$$

$$q_{t+1}(s, a) = q_t(s, a), \quad \forall (s, a) \neq (s_t, a_t),$$

其中$t = 0, 1, 2, \ldots$。

- $q_t(s_t, a_t)$ 是对$q_\pi(s_t, a_t)$ 的估计;
- $\alpha_t(s_t, a_t)$ 是依赖于s_t, a_t 的学习率。

Sarsa——算法

首先，我们的目标是估计给定策略π 的动作值。

假设我们有一些经验$\{(s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1})\}_t$。
可以使用如下*Sarsa* 算法估计动作值:

$$q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha_t(s_t, a_t)\Big[q_t(s_t, a_t) - [r_{t+1} + \gamma q_t(s_{t+1}, a_{t+1})]\Big],$$

$$q_{t+1}(s, a) = q_t(s, a), \quad \forall (s, a) \neq (s_t, a_t),$$

其中$t = 0, 1, 2, \ldots$。

- $q_t(s_t, a_t)$ 是对$q_\pi(s_t, a_t)$ 的估计;
- $\alpha_t(s_t, a_t)$ 是依赖于s_t, a_t 的学习率。

Sarsa——算法

首先，我们的目标是估计给定策略π 的动作值。

假设我们有一些经验$\{(s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1})\}_t$。
可以使用如下*Sarsa* 算法估计动作值：

$$q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha_t(s_t, a_t)\Big[q_t(s_t, a_t) - [r_{t+1} + \gamma q_t(s_{t+1}, a_{t+1})]\Big],$$

$$q_{t+1}(s, a) = q_t(s, a), \quad \forall (s, a) \neq (s_t, a_t),$$

其中$t = 0, 1, 2, \ldots$。

- $q_t(s_t, a_t)$ 是对$q_\pi(s_t, a_t)$ 的估计；
- $\alpha_t(s_t, a_t)$ 是依赖于s_t, a_t 的学习率。

Sarsa——算法

- **为什么该算法称为Sarsa?** 因为算法的每一步涉及$(s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1})$。Sarsa 是"状态-动作-奖励-状态-动作"（state-action-reward-state-action）的缩写。
- **Sarsa 与之前TD 学习算法的关系是什么?** 将TD 算法中的状态值估计$v(s)$替换为动作值估计$q(s, a)$ 即可得到Sarsa。因此，Sarsa 是TD 算法的动作值版本。
- **Sarsa 算法在数学上做了什么?** Sarsa 的表达式说明它是求解如下方程的随机近似算法:

$$q_\pi(s, a) = \mathbb{E}\left[R + \gamma q_\pi(S', A') | s, a\right], \quad \forall s, a.$$

这是以动作值表示的Bellman 方程的另一种形式。证明见教材。

Sarsa——算法

- **为什么该算法称为Sarsa?** 因为算法的每一步涉及$(s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1})$。Sarsa 是"状态-动作-奖励-状态-动作"（state-action-reward-state-action）的缩写。
- **Sarsa 与之前TD 学习算法的关系是什么?** 将TD 算法中的状态值估计$v(s)$替换为动作值估计$q(s,a)$ 即可得到Sarsa。因此，Sarsa 是TD 算法的动作值版本。
- **Sarsa 算法在数学上做了什么?** Sarsa 的表达式说明它是求解如下方程的随机近似算法:

$$q_\pi(s,a) = \mathbb{E}\left[R + \gamma q_\pi(S', A') | s, a\right], \quad \forall s, a.$$

这是以动作值表示的Bellman 方程的另一种形式。证明见教材。

Sarsa——算法

- **为什么该算法称为Sarsa?** 因为算法的每一步涉及$(s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1})$。Sarsa 是"状态-动作-奖励-状态-动作"（state-action-reward-state-action）的缩写。
- **Sarsa 与之前TD 学习算法的关系是什么?** 将TD 算法中的状态值估计$v(s)$替换为动作值估计$q(s,a)$ 即可得到Sarsa。因此，Sarsa 是TD 算法的动作值版本。
- **Sarsa 算法在数学上做了什么?** Sarsa 的表达式说明它是求解如下方程的随机近似算法:

$$q_\pi(s,a) = \mathbb{E}\left[R + \gamma q_\pi(S', A')|s,a\right], \quad \forall s, a.$$

这是以动作值表示的Bellman 方程的另一种形式。证明见教材。

Sarsa——算法

Theorem (Sarsa 学习的收敛性)

由*Sarsa* 算法，若对所有(s,a) 满足$\sum_t \alpha_t(s,a)=\infty$ 且$\sum_t \alpha_t^2(s,a)<\infty$，则$q_t(s,a)$ 以概率*1* 收敛到动作值$q_\pi(s,a)$（$t\to\infty$）。

注:

- 该定理指出，对于给定策略π，Sarsa 可以求得动作值。

Sarsa——算法

Theorem (Sarsa 学习的收敛性)

由*Sarsa* 算法，若对所有(s,a) 满足$\sum_t \alpha_t(s,a)=\infty$ 且$\sum_t \alpha_t^2(s,a)<\infty$，则$q_t(s,a)$ 以概率*1* 收敛到动作值$q_\pi(s,a)$（$t\to\infty$）。

注：

- 该定理指出，对于给定策略π，Sarsa 可以求得动作值。

Sarsa——实现

强化学习的最终目标是寻找最优策略。
为此，可以将Sarsa 与策略改进步骤结合。
结合后的算法也称为Sarsa。

Sarsa——实现

强化学习的最终目标是寻找最优策略。

为此，可以将Sarsa 与策略改进步骤结合。

结合后的算法也称为Sarsa。

伪代码：Sarsa 策略搜索

对每个回合，执行：

在s_0 处按$\pi_0(s_0)$ 生成a_0

若s_t（$t = 0, 1, 2, \ldots$）不是目标状态，执行：

给定(s_t, a_t)，采集经验样本$(r_{t+1}, s_{t+1}, a_{t+1})$：与环境交互生成$r_{t+1}, s_{t+1}$；

按$\pi_t(s_{t+1})$ 生成a_{t+1}。

更新(s_t, a_t) 的Q 值：

$$q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha_t(s_t, a_t)\Big[q_t(s_t, a_t) - (r_{t+1} + \gamma q_t(s_{t+1}, a_{t+1}))\Big]$$

更新s_t 的策略：

若$a = \arg\max_a q_{t+1}(s_t, a)$，则$\pi_{t+1}(a|s_t) = 1 - \frac{\epsilon}{|\mathcal{A}(s_t)|}(|\mathcal{A}(s_t)| - 1)$

否则$\pi_{t+1}(a|s_t) = \frac{\epsilon}{|\mathcal{A}(s_t)|}$

$s_t \leftarrow s_{t+1}$，$a_t \leftarrow a_{t+1}$

Sarsa——实现

关于该算法的说明：

- 在更新$q(s_t, a_t)$ 后立即更新s_t 的策略，这基于广义策略迭代的思想。
- 策略采用ϵ-贪心而非贪心策略，以平衡利用与探索。

明确核心思想与复杂性来源：

- 核心思想很简单：即使用算法求解给定策略的Bellman 方程。
- 复杂性来源于我们试图高效地寻找最优策略这一目标。

Sarsa——实现

关于该算法的说明：

- 在更新$q(s_t, a_t)$ 后立即更新s_t 的策略，这基于广义策略迭代的思想。
- 策略采用ϵ-贪心而非贪心策略，以平衡利用与探索。

明确核心思想与复杂性来源：

- 核心思想很简单：即使用算法求解给定策略的Bellman 方程。
- 复杂性来源于我们试图高效地寻找最优策略这一目标。

Sarsa——示例

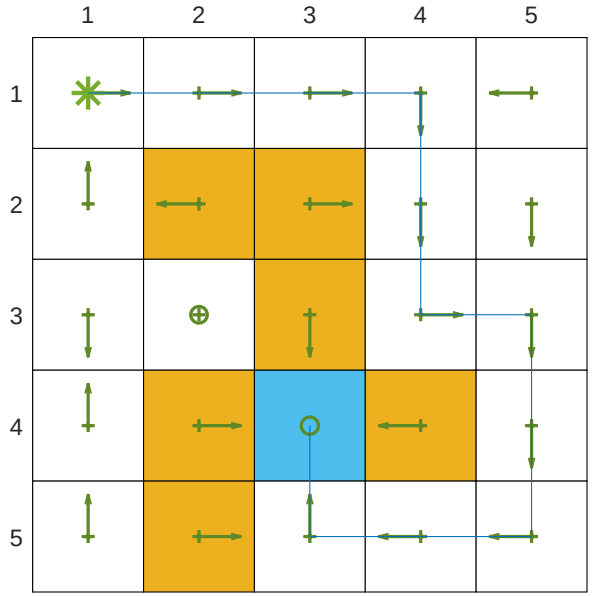
任务描述：

- 任务是从特定起始状态找到一条通往目标状态的路径。
  - 该任务与之前所有任务不同——之前的任务是为每个状态找到最优策略！
  - 每个回合从左上角状态开始，到达目标状态结束。
  - 以后请注意任务的具体设定。
- $r_{\text{target}} = 0$，$r_{\text{forbidden}} = r_{\text{boundary}} = -10$，$r_{\text{other}} = -1$。学习率$\alpha = 0.1$，$\epsilon = 0.1$。

Sarsa——示例

结果:

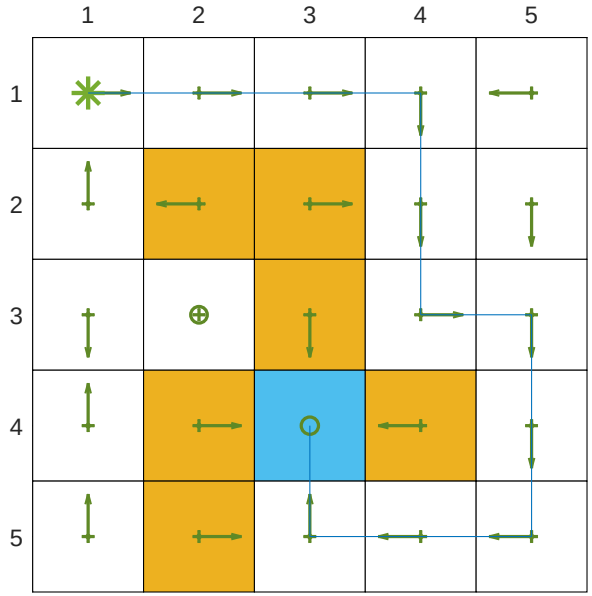
- 上方左图显示了Sarsa 得到的最终策略。
  - 并非所有状态都有最优策略。

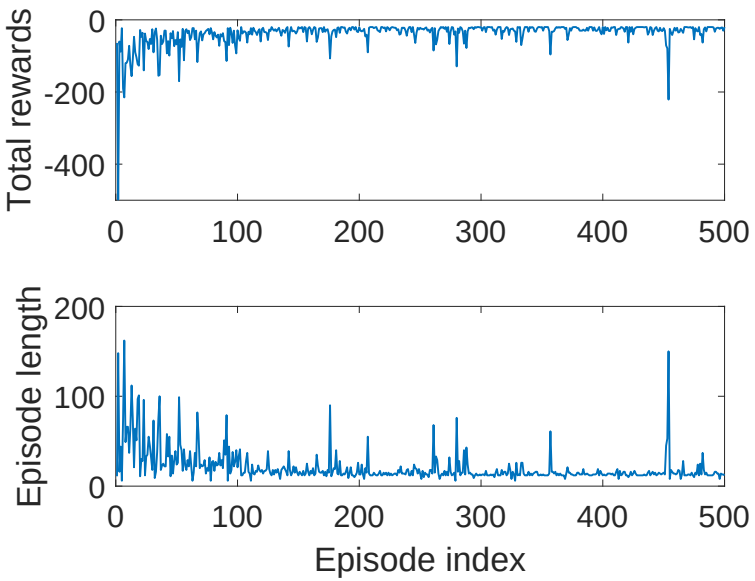


Sarsa——示例

结果:

- 上方左图显示了Sarsa 得到的最终策略。
  - 并非所有状态都有最优策略。
- 右图显示了每个回合的总奖励与长度。
  - 每个回合的总奖励将作为常用评估指标。





大纲

1 动机示例

2 状态值的TD 学习

3 动作值的TD 学习：Sarsa

4 动作值的TD 学习：n 步Sarsa

5 最优动作值的TD 学习：Q 学习

6 统一视角

7 总结

动作值的TD 学习：n 步Sarsa

n 步Sarsa 可以统一 Sarsa 与蒙特卡洛学习

动作值的定义为：

$$q_\pi(s,a) = \mathbb{E}[G_t | S_t = s, A_t = a].$$

折扣回报G_t 可以写成以下不同形式：

动作值的TD 学习：n 步Sarsa

n 步Sarsa 可以统一 Sarsa 与蒙特卡洛学习

动作值的定义为：

$$q_\pi(s,a) = \mathbb{E}[G_t | S_t = s, A_t = a].$$

折扣回报G_t 可以写成以下不同形式：

动作值的TD 学习：n 步Sarsa

n 步Sarsa 可以统一 Sarsa 与蒙特卡洛学习

动作值的定义为：

$$q_\pi(s,a) = \mathbb{E}[G_t | S_t = s, A_t = a].$$

折扣回报G_t 可以写成以下不同形式：

$$\text{Sarsa} \longleftarrow \quad G_t^{(1)} = R_{t+1} + \gamma q_\pi(S_{t+1}, A_{t+1}),$$

动作值的TD 学习：n 步Sarsa

n 步Sarsa 可以统一 Sarsa 与蒙特卡洛学习

动作值的定义为：

$$q_\pi(s,a) = \mathbb{E}[G_t | S_t = s, A_t = a].$$

折扣回报G_t 可以写成以下不同形式：

$$\text{Sarsa} \longleftarrow \quad G_t^{(1)} = R_{t+1} + \gamma q_\pi(S_{t+1}, A_{t+1}),$$
$$G_t^{(2)} = R_{t+1} + \gamma R_{t+2} + \gamma^2 q_\pi(S_{t+2}, A_{t+2}),$$

动作值的TD 学习：n 步Sarsa

n 步Sarsa 可以统一 Sarsa 与蒙特卡洛学习

动作值的定义为：

$$q_\pi(s,a) = \mathbb{E}[G_t | S_t = s, A_t = a].$$

折扣回报G_t 可以写成以下不同形式：

$$\begin{aligned}
\text{Sarsa} \longleftarrow \quad G_t^{(1)} &= R_{t+1} + \gamma q_\pi(S_{t+1}, A_{t+1}), \\
G_t^{(2)} &= R_{t+1} + \gamma R_{t+2} + \gamma^2 q_\pi(S_{t+2}, A_{t+2}), \\
&\vdots \\
n \text{ 步Sarsa} \longleftarrow \quad G_t^{(n)} &= R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^n q_\pi(S_{t+n}, A_{t+n}),
\end{aligned}$$

动作值的TD 学习：n 步Sarsa

n 步Sarsa 可以统一 Sarsa 与蒙特卡洛学习

动作值的定义为：

$$q_\pi(s,a) = \mathbb{E}[G_t | S_t = s, A_t = a].$$

折扣回报G_t 可以写成以下不同形式：

$$\begin{aligned}
\text{Sarsa} \longleftarrow \quad G_t^{(1)} &= R_{t+1} + \gamma q_\pi(S_{t+1}, A_{t+1}), \\
G_t^{(2)} &= R_{t+1} + \gamma R_{t+2} + \gamma^2 q_\pi(S_{t+2}, A_{t+2}), \\
&\vdots \\
n \text{ 步Sarsa} \longleftarrow \quad G_t^{(n)} &= R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^n q_\pi(S_{t+n}, A_{t+n}), \\
&\vdots \\
\text{MC} \longleftarrow \quad G_t^{(\infty)} &= R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \ldots
\end{aligned}$$

动作值的TD 学习：n 步Sarsa

n 步Sarsa 可以统一 Sarsa 与蒙特卡洛学习

动作值的定义为：

$$q_\pi(s,a) = \mathbb{E}[G_t | S_t = s, A_t = a].$$

折扣回报G_t 可以写成以下不同形式：

$$\begin{aligned}
\text{Sarsa} \longleftarrow \quad G_t^{(1)} &= R_{t+1} + \gamma q_\pi(S_{t+1}, A_{t+1}), \\
G_t^{(2)} &= R_{t+1} + \gamma R_{t+2} + \gamma^2 q_\pi(S_{t+2}, A_{t+2}), \\
&\vdots \\
n \text{ 步Sarsa} \longleftarrow \quad G_t^{(n)} &= R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^n q_\pi(S_{t+n}, A_{t+n}), \\
&\vdots \\
\text{MC} \longleftarrow \quad G_t^{(\infty)} &= R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \ldots
\end{aligned}$$

需要注意的是，$G_t = G_t^{(1)} = G_t^{(2)} = G_t^{(n)} = G_t^{(\infty)}$，上标仅表示$G_t$ 的不同分解结构。

动作值的TD 学习：n 步Sarsa

- Sarsa 旨在求解：

$$q_\pi(s,a) = \mathbb{E}[G_t^{(1)}|s,a] = \mathbb{E}[R_{t+1} + \gamma q_\pi(S_{t+1}, A_{t+1})|s,a].$$

- MC 学习旨在求解：

$$q_\pi(s,a) = \mathbb{E}[G_t^{(\infty)}|s,a] = \mathbb{E}[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \ldots|s,a].$$

- 介于两者之间的n 步*Sarsa* 旨在求解：

$$q_\pi(s,a) = \mathbb{E}[G_t^{(n)}|s,a] = \mathbb{E}[R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^n q_\pi(S_{t+n}, A_{t+n})|s,a].$$

- n 步Sarsa 的算法为：

$$\begin{aligned} q_{t+1}(s_t,a_t) = q_t(s_t,a_t) \\ - \alpha_t(s_t,a_t)\Big[q_t(s_t,a_t) - [r_{t+1} + \gamma r_{t+2} + \cdots + \gamma^n q_t(s_{t+n}, a_{t+n})]\Big]. \end{aligned}$$

  - 当$n = 1$ 时，n 步Sarsa 退化为（单步）Sarsa 算法。
  - 当$n = \infty$ 时，n 步Sarsa 退化为MC 学习算法。

动作值的TD 学习：n 步Sarsa

- Sarsa 旨在求解：

$$q_\pi(s,a) = \mathbb{E}[G_t^{(1)}|s,a] = \mathbb{E}[R_{t+1} + \gamma q_\pi(S_{t+1}, A_{t+1})|s,a].$$

- MC 学习旨在求解：

$$q_\pi(s,a) = \mathbb{E}[G_t^{(\infty)}|s,a] = \mathbb{E}[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \ldots|s,a].$$

- 介于两者之间的n 步*Sarsa* 旨在求解：

$$q_\pi(s,a) = \mathbb{E}[G_t^{(n)}|s,a] = \mathbb{E}[R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^n q_\pi(S_{t+n}, A_{t+n})|s,a].$$

- n 步Sarsa 的算法为：

$$q_{t+1}(s_t,a_t) = q_t(s_t,a_t) - \alpha_t(s_t,a_t)\Big[q_t(s_t,a_t) - [r_{t+1} + \gamma r_{t+2} + \cdots + \gamma^n q_t(s_{t+n}, a_{t+n})]\Big].$$

  - 当$n=1$ 时，n 步Sarsa 退化为（单步）Sarsa 算法。
  - 当$n=\infty$ 时，n 步Sarsa 退化为MC 学习算法。

动作值的TD 学习：n 步Sarsa

- Sarsa 旨在求解：

$$q_\pi(s,a) = \mathbb{E}[G_t^{(1)}|s,a] = \mathbb{E}[R_{t+1} + \gamma q_\pi(S_{t+1}, A_{t+1})|s,a].$$

- MC 学习旨在求解：

$$q_\pi(s,a) = \mathbb{E}[G_t^{(\infty)}|s,a] = \mathbb{E}[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \ldots|s,a].$$

- 介于两者之间的n 步*Sarsa* 旨在求解：

$$q_\pi(s,a) = \mathbb{E}[G_t^{(n)}|s,a] = \mathbb{E}[R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^n q_\pi(S_{t+n}, A_{t+n})|s,a].$$

- n 步Sarsa 的算法为：

$$q_{t+1}(s_t,a_t) = q_t(s_t,a_t) - \alpha_t(s_t,a_t)\Big[q_t(s_t,a_t) - [r_{t+1} + \gamma r_{t+2} + \cdots + \gamma^n q_t(s_{t+n},a_{t+n})]\Big].$$

  - 当$n=1$ 时，n 步Sarsa 退化为（单步）Sarsa 算法。
  - 当$n=\infty$ 时，n 步Sarsa 退化为MC 学习算法。

动作值的TD 学习：n 步Sarsa

- Sarsa 旨在求解：

$$q_\pi(s,a) = \mathbb{E}[G_t^{(1)}|s,a] = \mathbb{E}[R_{t+1} + \gamma q_\pi(S_{t+1}, A_{t+1})|s,a].$$

- MC 学习旨在求解：

$$q_\pi(s,a) = \mathbb{E}[G_t^{(\infty)}|s,a] = \mathbb{E}[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \ldots|s,a].$$

- 介于两者之间的n 步*Sarsa* 旨在求解：

$$q_\pi(s,a) = \mathbb{E}[G_t^{(n)}|s,a] = \mathbb{E}[R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^n q_\pi(S_{t+n}, A_{t+n})|s,a].$$

- n 步Sarsa 的算法为：

$$\begin{aligned} q_{t+1}(s_t,a_t) = q_t(s_t,a_t) \\ - \alpha_t(s_t,a_t)\Big[q_t(s_t,a_t) - [r_{t+1} + \gamma r_{t+2} + \cdots + \gamma^n q_t(s_{t+n}, a_{t+n})]\Big]. \end{aligned}$$

  - 当$n = 1$ 时，n 步Sarsa 退化为（单步）Sarsa 算法。
  - 当$n = \infty$ 时，n 步Sarsa 退化为MC 学习算法。

动作值的TD 学习：n 步Sarsa

- 数据：n 步Sarsa 需要$(s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1}, \ldots, r_{t+n}, s_{t+n}, a_{t+n})$。
- 由于$(r_{t+n}, s_{t+n}, a_{t+n})$ 在时刻t 尚未收集，我们无法在t 时刻实现n 步Sarsa。需要等到时刻$t+n$ 才能更新(s_t, a_t) 的Q 值：

$$q_{t+n}(s_t, a_t) = q_{t+n-1}(s_t, a_t)$$
$$- \alpha_{t+n-1}(s_t, a_t)\Big[q_{t+n-1}(s_t, a_t) - [r_{t+1} + \gamma r_{t+2} + \cdots + \gamma^n q_{t+n-1}(s_{t+n}, a_{t+n})]\Big]$$

- 由于n 步Sarsa 将Sarsa 和MC 学习作为两个极端情况包含在内，其性能是两者的融合：
  - 若n 较大，其性能接近MC 学习，方差大、偏差小。
  - 若n 较小，其性能接近Sarsa，由于初始猜测导致偏差相对较大，但方差相对较低。
- 最后，n 步Sarsa 同样用于策略评估，可与策略改进步骤结合以搜索最优策略。

动作值的TD 学习：n 步Sarsa

- 数据：n 步Sarsa 需要$(s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1}, \ldots, r_{t+n}, s_{t+n}, a_{t+n})$。
- 由于$(r_{t+n}, s_{t+n}, a_{t+n})$ 在时刻t 尚未收集，我们无法在t 时刻实现n 步Sarsa。需要等到时刻$t+n$ 才能更新(s_t, a_t) 的Q 值：

$$q_{t+n}(s_t, a_t) = q_{t+n-1}(s_t, a_t)$$
$$- \alpha_{t+n-1}(s_t, a_t)\Big[q_{t+n-1}(s_t, a_t) - [r_{t+1} + \gamma r_{t+2} + \cdots + \gamma^n q_{t+n-1}(s_{t+n}, a_{t+n})]\Big]$$

- 由于n 步Sarsa 将Sarsa 和MC 学习作为两个极端情况包含在内，其性能是两者的融合：
  - 若n 较大，其性能接近MC 学习，方差大、偏差小。
  - 若n 较小，其性能接近Sarsa，由于初始猜测导致偏差相对较大，但方差相对较低。
- 最后，n 步Sarsa 同样用于策略评估，可与策略改进步骤结合以搜索最优策略。

动作值的TD 学习：n 步Sarsa

- 数据：n 步Sarsa 需要$(s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1}, \ldots, r_{t+n}, s_{t+n}, a_{t+n})$。
- 由于$(r_{t+n}, s_{t+n}, a_{t+n})$ 在时刻t 尚未收集，我们无法在t 时刻实现n 步Sarsa。需要等到时刻$t+n$ 才能更新(s_t, a_t) 的Q 值：

$$q_{t+n}(s_t, a_t) = q_{t+n-1}(s_t, a_t)$$
$$- \alpha_{t+n-1}(s_t, a_t)\Big[q_{t+n-1}(s_t, a_t) - [r_{t+1} + \gamma r_{t+2} + \cdots + \gamma^n q_{t+n-1}(s_{t+n}, a_{t+n})]\Big]$$

- 由于n 步Sarsa 将Sarsa 和MC 学习作为两个极端情况包含在内，其性能是两者的融合：
  - 若n 较大，其性能接近MC 学习，方差大、偏差小。
  - 若n 较小，其性能接近Sarsa，由于初始猜测导致偏差相对较大，但方差相对较低。
- 最后，n 步Sarsa 同样用于策略评估，可与策略改进步骤结合以搜索最优策略。

动作值的TD 学习：n 步Sarsa

- 数据：n 步Sarsa 需要$(s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1}, \ldots, r_{t+n}, s_{t+n}, a_{t+n})$。
- 由于$(r_{t+n}, s_{t+n}, a_{t+n})$ 在时刻t 尚未收集，我们无法在t 时刻实现n 步Sarsa。需要等到时刻$t+n$ 才能更新(s_t, a_t) 的Q 值：

$$q_{t+n}(s_t, a_t) = q_{t+n-1}(s_t, a_t)$$
$$- \alpha_{t+n-1}(s_t, a_t)\Big[q_{t+n-1}(s_t, a_t) - [r_{t+1} + \gamma r_{t+2} + \cdots + \gamma^n q_{t+n-1}(s_{t+n}, a_{t+n})]\Big]$$

- 由于n 步Sarsa 将Sarsa 和MC 学习作为两个极端情况包含在内，其性能是两者的融合：
  - 若n 较大，其性能接近MC 学习，方差大、偏差小。
  - 若n 较小，其性能接近Sarsa，由于初始猜测导致偏差相对较大，但方差相对较低。
- 最后，n 步Sarsa 同样用于策略评估，可与策略改进步骤结合以搜索最优策略。

动作值的TD 学习：n 步Sarsa

- 数据：n 步Sarsa 需要$(s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1}, \ldots, r_{t+n}, s_{t+n}, a_{t+n})$。
- 由于$(r_{t+n}, s_{t+n}, a_{t+n})$ 在时刻t 尚未收集，我们无法在t 时刻实现n 步Sarsa。需要等到时刻$t+n$ 才能更新(s_t, a_t) 的Q 值：

$$q_{t+n}(s_t, a_t) = q_{t+n-1}(s_t, a_t)$$
$$- \alpha_{t+n-1}(s_t, a_t)\Big[q_{t+n-1}(s_t, a_t) - [r_{t+1} + \gamma r_{t+2} + \cdots + \gamma^n q_{t+n-1}(s_{t+n}, a_{t+n})]\Big]$$

- 由于n 步Sarsa 将Sarsa 和MC 学习作为两个极端情况包含在内，其性能是两者的融合：
  - 若n 较大，其性能接近MC 学习，方差大、偏差小。
  - 若n 较小，其性能接近Sarsa，由于初始猜测导致偏差相对较大，但方差相对较低。
- 最后，n 步Sarsa 同样用于策略评估，可与策略改进步骤结合以搜索最优策略。

动作值的TD 学习：n 步Sarsa

- 数据：n 步Sarsa 需要$(s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1}, \ldots, r_{t+n}, s_{t+n}, a_{t+n})$。
- 由于$(r_{t+n}, s_{t+n}, a_{t+n})$ 在时刻t 尚未收集，我们无法在t 时刻实现n 步Sarsa。需要等到时刻$t+n$ 才能更新(s_t, a_t) 的Q 值：

$$q_{t+n}(s_t, a_t) = q_{t+n-1}(s_t, a_t)$$
$$- \alpha_{t+n-1}(s_t, a_t)\Big[q_{t+n-1}(s_t, a_t) - [r_{t+1} + \gamma r_{t+2} + \cdots + \gamma^n q_{t+n-1}(s_{t+n}, a_{t+n})]\Big]$$

- 由于n 步Sarsa 将Sarsa 和MC 学习作为两个极端情况包含在内，其性能是两者的融合：
  - 若n 较大，其性能接近MC 学习，方差大、偏差小。
  - 若n 较小，其性能接近Sarsa，由于初始猜测导致偏差相对较大，但方差相对较低。
- 最后，n 步Sarsa 同样用于策略评估，可与策略改进步骤结合以搜索最优策略。

大纲

1 动机示例

2 状态值的TD 学习

3 动作值的TD 学习：Sarsa

4 动作值的TD 学习：n 步Sarsa

5 最优动作值的TD 学习：Q 学习

6 统一视角

7 总结

最优动作值的TD 学习：Q 学习

- 接下来，我们介绍Q 学习，这是最广泛使用的强化学习算法之一。
- Sarsa 可以估计给定策略的动作值，必须结合策略改进步骤才能寻找最优策略。
- Q 学习可以直接估计最优动作值，从而得到最优策略。

Q 学习——算法

Q 学习算法为:

$$q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha_t(s_t, a_t)\left[q_t(s_t, a_t) - [r_{t+1} + \gamma \max_{a \in \mathcal{A}} q_t(s_{t+1}, a)]\right],$$

$$q_{t+1}(s, a) = q_t(s, a), \quad \forall (s, a) \neq (s_t, a_t),$$

Q 学习与Sarsa 非常相似，区别仅在于TD 目标:

- Q 学习的TD 目标为$r_{t+1} + \gamma \max_{a \in \mathcal{A}} q_t(s_{t+1}, a)$
- Sarsa 的TD 目标为$r_{t+1} + \gamma q_t(s_{t+1}, a_{t+1})$

Q 学习——算法

Q 学习算法为：

$$q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha_t(s_t, a_t)\left[q_t(s_t, a_t) - [r_{t+1} + \gamma \max_{a \in \mathcal{A}} q_t(s_{t+1}, a)]\right],$$

$$q_{t+1}(s, a) = q_t(s, a), \quad \forall (s, a) \neq (s_t, a_t),$$

Q 学习与Sarsa 非常相似，区别仅在于TD 目标：

- Q 学习的TD 目标为$r_{t+1} + \gamma \max_{a \in \mathcal{A}} q_t(s_{t+1}, a)$
- Sarsa 的TD 目标为$r_{t+1} + \gamma q_t(s_{t+1}, a_{t+1})$

Q 学习——算法

Q 学习在数学上做了什么?
它旨在求解:

$$q(s,a) = \mathbb{E}\left[R_{t+1} + \gamma \max_a q(S_{t+1}, a) \middle| S_t = s, A_t = a\right], \quad \forall s, a.$$

这是以动作值表示的Bellman 最优方程。证明见教材。

因此，Q 学习可以直接估计最优动作值，而非给定策略的动作值。

离策略与同策略

在深入研究Q 学习之前，先介绍两个重要概念：同策略学习与离策略学习。

TD 学习任务中存在两种策略：

- 行为策略用于生成经验样本。
- 目标策略不断朝最优策略更新。

同策略与离策略：

- 当行为策略与目标策略相同时，称为同策略（on-policy）学习。
- 当两者不同时，称为离策略（off-policy）学习。

离策略与同策略

在深入研究Q 学习之前，先介绍两个重要概念：同策略学习与离策略学习。

TD 学习任务中存在两种策略：

- 行为策略用于生成经验样本。
- 目标策略不断朝最优策略更新。

同策略与离策略：

- 当行为策略与目标策略相同时，称为同策略（on-policy）学习。
- 当两者不同时，称为离策略（off-policy）学习。

离策略与同策略

在深入研究Q 学习之前，先介绍两个重要概念：同策略学习与离策略学习。

TD 学习任务中存在两种策略：

- 行为策略用于生成经验样本。
- 目标策略不断朝最优策略更新。

同策略与离策略：

- 当行为策略与目标策略相同时，称为同策略（on-policy）学习。
- 当两者不同时，称为离策略（off-policy）学习。

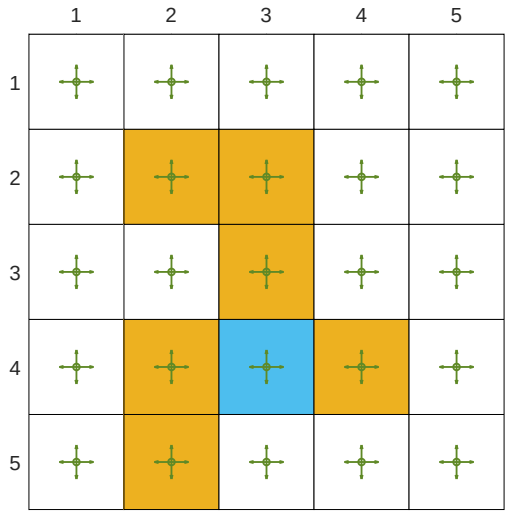
离策略与同策略

离策略学习的优势：

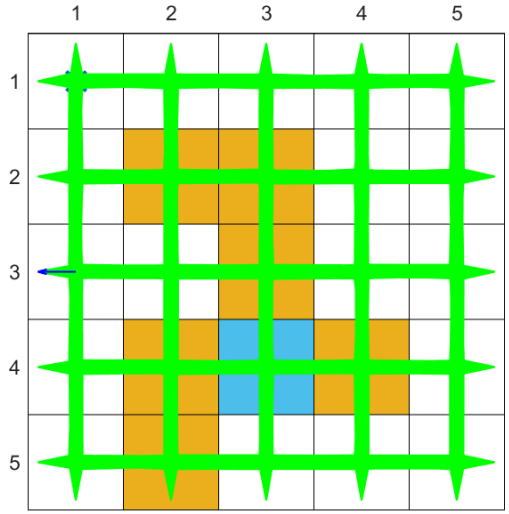
- 可以基于任意其他策略生成的经验样本搜索最优策略。

离策略学习的优势：

- 可以基于任意其他策略生成的经验样本搜索最优策略。
- 示例：行为策略具有探索性，从而可以生成覆盖每个状态-动作对足够多次的回合。



(a) 探索性行为策略



(b) 生成的回合

如何判断一个TD 算法是同策略还是离策略?

- 首先，检查算法旨在求解什么数学问题。
- 其次，检查算法需要什么经验样本。

这一点值得特别关注，因为对初学者来说容易混淆。

离策略与同策略

- Sarsa 旨在通过求解以下方程评估给定策略π:

$$q_\pi(s,a) = \mathbb{E}\left[R + \gamma q_\pi(S', A')|s,a\right], \quad \forall s,a.$$

其中$R \sim p(R|s,a)$，$S' \sim p(S'|s,a)$，$A' \sim \pi(A'|S')$。

- MC 旨在通过求解以下方程评估给定策略π:

$$q_\pi(s,a) = \mathbb{E}\left[R_{t+1} + \gamma R_{t+2} + \ldots | S_t = s, A_t = a\right], \quad \forall s,a.$$

其中样本由π 生成。

Sarsa 和MC 均为同策略。

- π 是行为策略，因为我们需要由π 生成的经验样本来估计π 的动作值。
- π 也是目标策略，因为它不断被更新以趋近于最优策略。

离策略与同策略

- Sarsa 旨在通过求解以下方程评估给定策略π:

$$q_\pi(s,a) = \mathbb{E}\left[R + \gamma q_\pi(S', A')|s,a\right], \quad \forall s,a.$$

其中$R \sim p(R|s,a)$，$S' \sim p(S'|s,a)$，$A' \sim \pi(A'|S')$。

- MC 旨在通过求解以下方程评估给定策略π:

$$q_\pi(s,a) = \mathbb{E}\left[R_{t+1} + \gamma R_{t+2} + \ldots | S_t = s, A_t = a\right], \quad \forall s,a.$$

其中样本由π 生成。

Sarsa 和MC 均为同策略。

- π 是行为策略，因为我们需要由π 生成的经验样本来估计π 的动作值。
- π 也是目标策略，因为它不断被更新以趋近于最优策略。

离策略与同策略

- Sarsa 旨在通过求解以下方程评估给定策略π:

$$q_\pi(s,a) = \mathbb{E}\left[R + \gamma q_\pi(S', A')|s,a\right], \quad \forall s,a.$$

其中$R \sim p(R|s,a)$，$S' \sim p(S'|s,a)$，$A' \sim \pi(A'|S')$。

- MC 旨在通过求解以下方程评估给定策略π:

$$q_\pi(s,a) = \mathbb{E}\left[R_{t+1} + \gamma R_{t+2} + \ldots | S_t = s, A_t = a\right], \quad \forall s,a.$$

其中样本由π 生成。

Sarsa 和MC 均为同策略。

- π 是行为策略，因为我们需要由π 生成的经验样本来估计π 的动作值。
- π 也是目标策略，因为它不断被更新以趋近于最优策略。

离策略与同策略

Q 学习是离策略的。

- 首先，Q 学习旨在求解Bellman 最优方程：

$$q(s,a) = \mathbb{E}\left[R_{t+1} + \gamma \max_a q(S_{t+1},a) \middle| S_t = s, A_t = a\right], \quad \forall s, a.$$

- 其次，算法为：

$$q_{t+1}(s_t,a_t) = q_t(s_t,a_t) - \alpha_t(s_t,a_t)\left[q_t(s_t,a_t) - [r_{t+1} + \gamma \max_{a\in\mathcal{A}} q_t(s_{t+1},a)]\right]$$

需要$(s_t, a_t, r_{t+1}, s_{t+1})$。

- 行为策略是在s_t 处生成a_t 所用的策略，可以是任意策略。

离策略与同策略

Q 学习是离策略的。

- 首先，Q 学习旨在求解Bellman 最优方程：

$$q(s,a) = \mathbb{E}\left[R_{t+1} + \gamma \max_a q(S_{t+1},a) \middle| S_t = s, A_t = a\right], \quad \forall s, a.$$

- 其次，算法为：

$$q_{t+1}(s_t,a_t) = q_t(s_t,a_t) - \alpha_t(s_t,a_t)\left[q_t(s_t,a_t) - [r_{t+1} + \gamma \max_{a\in\mathcal{A}} q_t(s_{t+1},a)]\right]$$

需要$(s_t, a_t, r_{t+1}, s_{t+1})$。

- 行为策略是在s_t 处生成a_t 所用的策略，可以是任意策略。

离策略与同策略

Q 学习是离策略的。

- 首先，Q 学习旨在求解Bellman 最优方程：

$$q(s,a) = \mathbb{E}\left[R_{t+1} + \gamma \max_a q(S_{t+1},a) \middle| S_t = s, A_t = a\right], \quad \forall s, a.$$

- 其次，算法为：

$$q_{t+1}(s_t,a_t) = q_t(s_t,a_t) - \alpha_t(s_t,a_t)\left[q_t(s_t,a_t) - [r_{t+1} + \gamma \max_{a\in\mathcal{A}} q_t(s_{t+1},a)]\right]$$

需要$(s_t, a_t, r_{t+1}, s_{t+1})$。

- 行为策略是在s_t 处生成a_t 所用的策略，可以是任意策略。

离策略与同策略

Q 学习是离策略的。

- 首先，Q 学习旨在求解Bellman 最优方程：

$$q(s,a) = \mathbb{E}\left[R_{t+1} + \gamma \max_a q(S_{t+1},a) \middle| S_t = s, A_t = a\right], \quad \forall s, a.$$

- 其次，算法为：

$$q_{t+1}(s_t,a_t) = q_t(s_t,a_t) - \alpha_t(s_t,a_t)\left[q_t(s_t,a_t) - [r_{t+1} + \gamma \max_{a\in\mathcal{A}} q_t(s_{t+1},a)]\right]$$

需要$(s_t, a_t, r_{t+1}, s_{t+1})$。

- 行为策略是在s_t 处生成a_t 所用的策略，可以是任意策略。

Q 学习——实现

由于Q 学习是离策略的，它可以以离策略或同策略的方式实现。

伪代码：Q 学习策略搜索（同策略版本）

对每个回合，执行：

若s_t（$t = 0, 1, 2, \ldots$）不是目标状态，执行：

给定s_t，采集经验样本(a_t, r_{t+1}, s_{t+1})：按$\pi_t(s_t)$ 生成a_t；与环境交互生成r_{t+1}, s_{t+1}。

更新(s_t, a_t) 的Q 值：

$$q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha_t(s_t, a_t)\Big[q_t(s_t, a_t) - (r_{t+1} + \gamma \max_a q_t(s_{t+1}, a))\Big]$$

更新s_t 的策略：

若$a = \arg\max_a q_{t+1}(s_t, a)$，则$\pi_{t+1}(a|s_t) = 1 - \frac{\epsilon}{|\mathcal{A}(s_t)|}(|\mathcal{A}(s_t)| - 1)$

否则$\pi_{t+1}(a|s_t) = \frac{\epsilon}{|\mathcal{A}(s_t)|}$

详细伪代码见教材。

Q 学习——算法

伪代码：Q 学习最优策略搜索（离策略版本）

目标： 利用由π_b 生成的经验样本，为所有状态学习最优目标策略π_T。

对由π_b 生成的每个回合$\{s_0, a_0, r_1, s_1, a_1, r_2, \ldots\}$，执行：

对回合的每个步骤$t = 0, 1, 2, \ldots$，执行：

更新(s_t, a_t) 的Q 值：

$$q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha_t(s_t, a_t)\Big[q(s_t, a_t) - (r_{t+1} + \gamma \max_a q_t(s_{t+1}, a))\Big]$$

更新s_t 的目标策略：

若$a = \arg\max_a q_{t+1}(s_t, a)$，则$\pi_{T,t+1}(a|s_t) = 1$

否则$\pi_{T,t+1}(a|s_t) = 0$

详细伪代码见教材。

Q 学习——示例

任务描述：

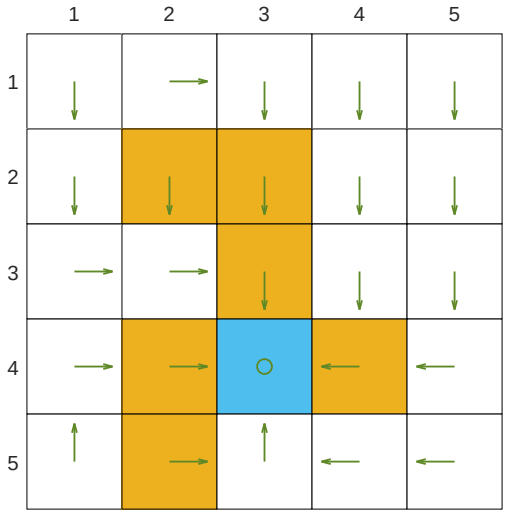
- 这些示例中的任务是为所有状态找到最优策略。
- 奖励设置：$r_{\text{boundary}} = r_{\text{forbidden}} = -1$，$r_{\text{target}} = 1$。折扣率$\gamma = 0.9$，学习率$\alpha = 0.1$。

Q 学习——示例

任务描述：

- 这些示例中的任务是为所有状态找到最优策略。
- 奖励设置：$r_{\text{boundary}} = r_{\text{forbidden}} = -1$，$r_{\text{target}} = 1$。折扣率$\gamma = 0.9$，学习率$\alpha = 0.1$。

真实值： 最优策略及对应的最优状态值。



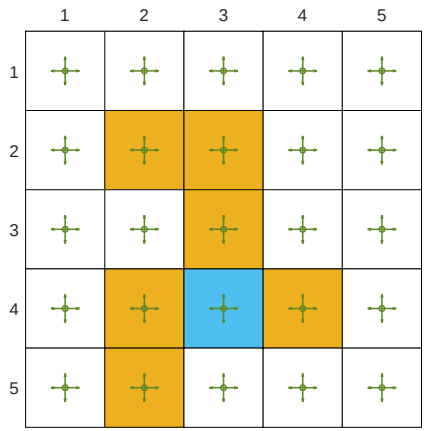
(a) 最优策略

	1	2	3	4	5
1	5.8	5.6	6.2	6.5	5.8
2	6.5	7.2	8.0	7.2	6.5
3	7.2	8.0	10.0	8.0	7.2
4	8.0	10.0	10.0	10.0	8.0
5	7.2	9.0	10.0	9.0	8.1

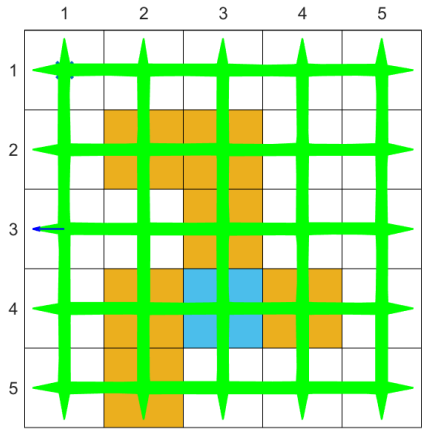
(b) 最优状态值

Q 学习——示例

行为策略与生成的经验（10^5 步）：



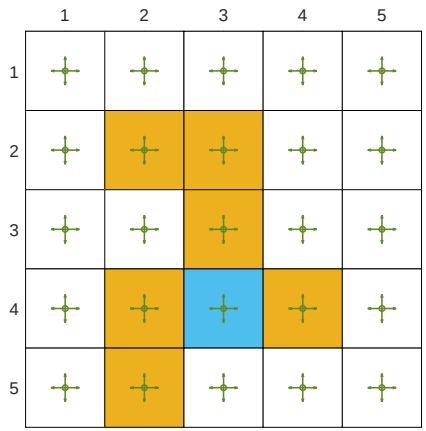
(a) 行为策略



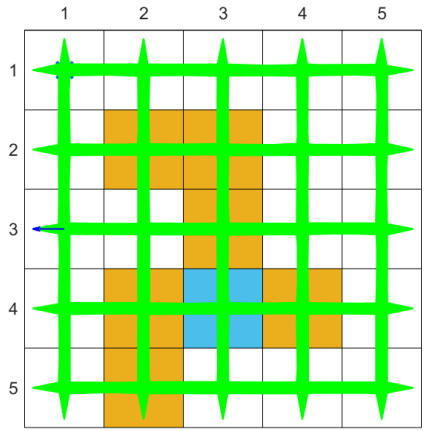
(b) 生成的回合

Q 学习——示例

行为策略与生成的经验（10^5 步）：

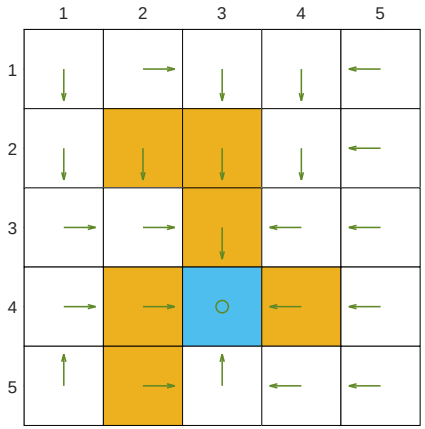


(a) 行为策略

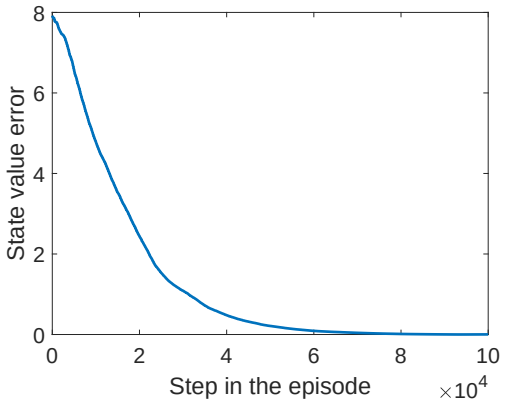


(b) 生成的回合

离策略Q 学习得到的策略:



(a) 估计策略

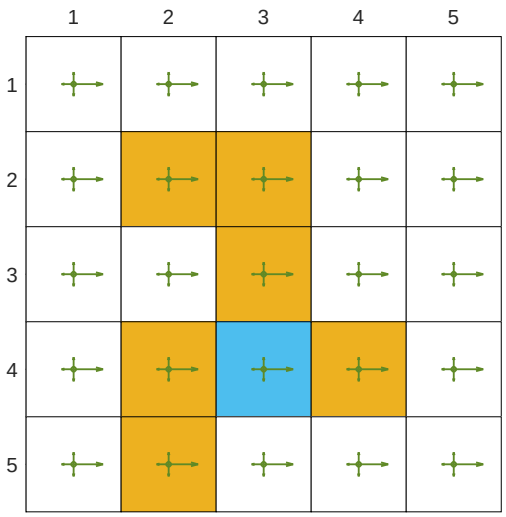


(b) 状态值误差

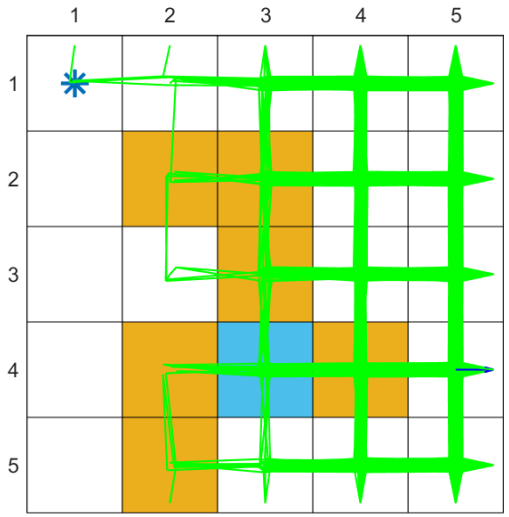
Q 学习——示例

探索的重要性：10^5 步的回合

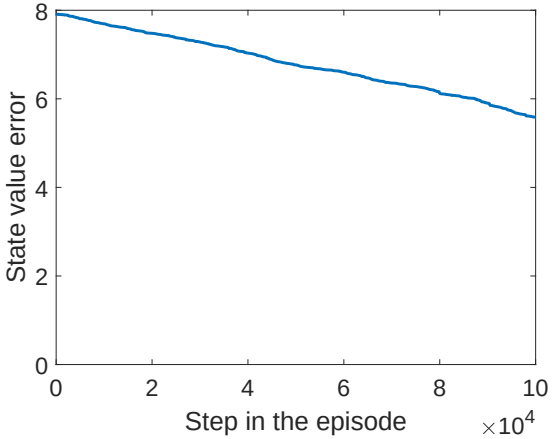
若策略探索性不足，样本质量较差。



(a) 行为策略$\epsilon = 0.5$

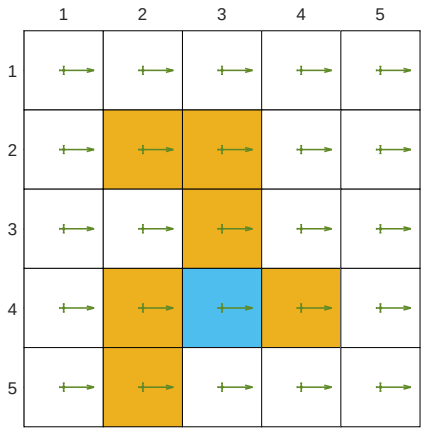


(b) 生成的回合

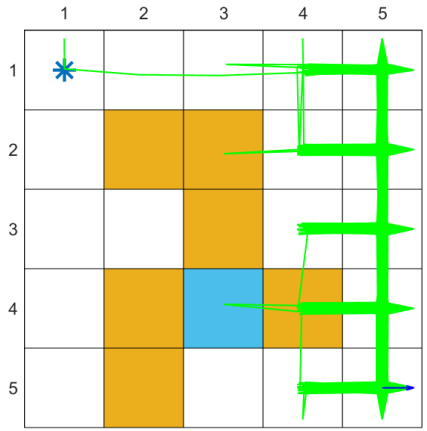


(c) Q 学习结果

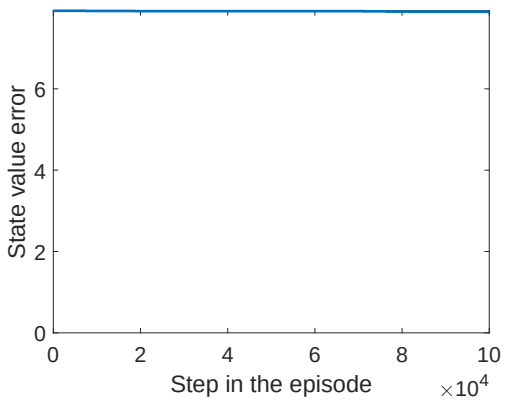
Q 学习——示例



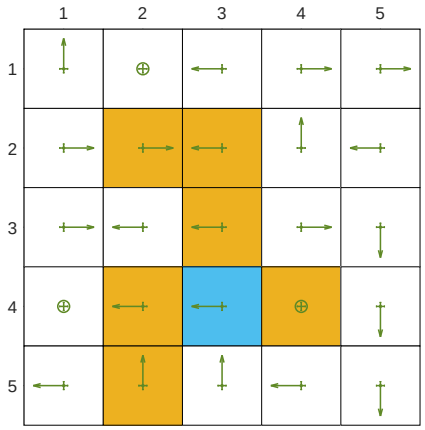
(a) 行为策略$\epsilon = 0.1$



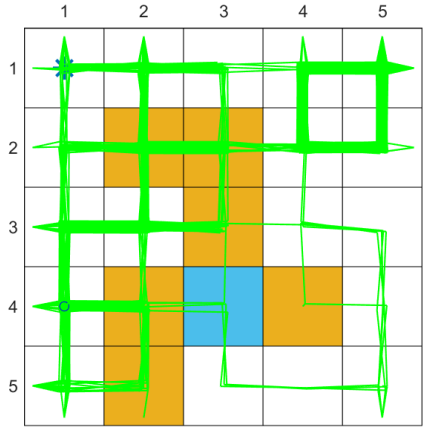
(b) 生成的回合



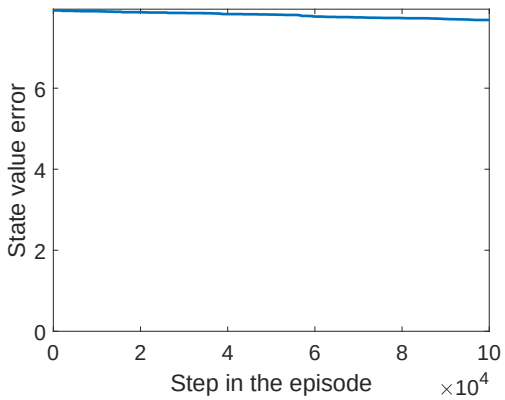
(c) Q 学习结果



(a) 行为策略$\epsilon = 0.1$



(b) 生成的回合



(c) Q 学习结果

大纲

1 动机示例

2 状态值的TD 学习

3 动作值的TD 学习：Sarsa

4 动作值的TD 学习：n 步Sarsa

5 最优动作值的TD 学习：Q 学习

6 统一视角

7 总结

统一视角

本讲介绍的所有算法均可以用一个统一的表达式表示:

$$q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha_t(s_t, a_t)[q_t(s_t, a_t) - \bar{q}_t]$$

其中$\bar{q}_t$ 为*TD* 目标。

不同的TD 算法对应不同的$\bar{q}_t$。

算法	$\bar{q}_t$ 的表达式
Sarsa	$\bar{q}_t = r_{t+1} + \gamma q_t(s_{t+1}, a_{t+1})$
n 步Sarsa	$\bar{q}_t = r_{t+1} + \gamma r_{t+2} + \cdots + \gamma^n q_t(s_{t+n}, a_{t+n})$
Q 学习	$\bar{q}_t = r_{t+1} + \gamma \max_a q_t(s_{t+1}, a)$
蒙特卡洛	$\bar{q}_t = r_{t+1} + \gamma r_{t+2} + \cdots$

注：MC 方法也可以用该统一表达式表示，令$\alpha_t(s_t, a_t) = 1$，
即$q_{t+1}(s_t, a_t) = \bar{q}_t$。

统一视角

本讲介绍的所有算法均可以用一个统一的表达式表示:

$$q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha_t(s_t, a_t)[q_t(s_t, a_t) - \bar{q}_t]$$

其中$\bar{q}_t$ 为*TD* 目标。

不同的TD 算法对应不同的$\bar{q}_t$。

算法	$\bar{q}_t$ 的表达式
Sarsa	$\bar{q}_t = r_{t+1} + \gamma q_t(s_{t+1}, a_{t+1})$
n 步Sarsa	$\bar{q}_t = r_{t+1} + \gamma r_{t+2} + \cdots + \gamma^n q_t(s_{t+n}, a_{t+n})$
Q 学习	$\bar{q}_t = r_{t+1} + \gamma \max_a q_t(s_{t+1}, a)$
蒙特卡洛	$\bar{q}_t = r_{t+1} + \gamma r_{t+2} + \ldots$

注：MC 方法也可以用该统一表达式表示，令$\alpha_t(s_t, a_t) = 1$，
即$q_{t+1}(s_t, a_t) = \bar{q}_t$。

统一视角

本讲介绍的所有算法均可以用一个统一的表达式表示:

$$q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha_t(s_t, a_t)[q_t(s_t, a_t) - \bar{q}_t]$$

其中$\bar{q}_t$ 为*TD* 目标。

不同的TD 算法对应不同的$\bar{q}_t$。

算法	$\bar{q}_t$ 的表达式
Sarsa	$\bar{q}_t = r_{t+1} + \gamma q_t(s_{t+1}, a_{t+1})$
n 步Sarsa	$\bar{q}_t = r_{t+1} + \gamma r_{t+2} + \cdots + \gamma^n q_t(s_{t+n}, a_{t+n})$
Q 学习	$\bar{q}_t = r_{t+1} + \gamma \max_a q_t(s_{t+1}, a)$
蒙特卡洛	$\bar{q}_t = r_{t+1} + \gamma r_{t+2} + \ldots$

注：MC 方法也可以用该统一表达式表示，令$\alpha_t(s_t, a_t) = 1$，
即$q_{t+1}(s_t, a_t) = \bar{q}_t$。

统一视角

所有TD 算法均可视为求解Bellman 方程或Bellman 最优方程的随机近似算法:

算法	所求解的方程
Sarsa	BE: $q_\pi(s,a) = \mathbb{E}\left[R_{t+1} + \gamma q_\pi(S_{t+1}, A_{t+1}) \middle| S_t = s, A_t = a\right]$
n 步Sarsa	BE: $q_\pi(s,a) = \mathbb{E}[R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^n q_\pi(s_{t+n}, a_{t+n}) \middle| S_t = s, A_t = a]$
Q 学习	BOE: $q(s,a) = \mathbb{E}\left[R_{t+1} + \gamma \max_a q(S_{t+1}, a) \middle| S_t = s, A_t = a\right]$
蒙特卡洛	BE: $q_\pi(s,a) = \mathbb{E}[R_{t+1} + \gamma R_{t+2} + \ldots \middle| S_t = s, A_t = a]$

大纲

1 动机示例

2 状态值的TD 学习

3 动作值的TD 学习：Sarsa

4 动作值的TD 学习：n 步Sarsa

5 最优动作值的TD 学习：Q 学习

6 统一视角

7 总结

总结

- 介绍了多种TD 学习算法
- 涵盖其表达式、数学解释、实现方式、相互关系及示例
- 提供了统一视角